Lecture 3: Vector Calculus and Optimization

Math for Machine Learning

October 23, 2026

Contents

Introduction **2**

1 Multivariable Functions **3**
1.1 Graphs and level sets 4
1.2 Limits and continuity 4

2 Derivatives of Functions of One Variable **5**
2.1 Taylor polynomials 6

3 Partial Derivatives and Gradients **8**
3.1 Differentiability 9
3.2 Directional derivatives and steepest ascent 10

4 Jacobians: Derivatives of Vector-Valued Functions **11**
4.1 The Jacobian determinant 12

5 The Chain Rule **13**

6 Gradients with Respect to Matrices **16**

7 Backpropagation and Automatic Differentiation **18**
7.1 Forward mode and reverse mode 19
7.2 Computation graphs 20
7.3 Backpropagation in a neural network 21

8 Higher Derivatives and Taylor's Theorem **22**
8.1 Taylor's theorem in several variables 23

9 Unconstrained Optimization **24**
9.1 The second-order test 25

10 Gradient Descent and Newton's Method **28**
10.1 Convergence for smooth functions 28
10.2 Gradient descent on quadratic functions 29
10.3 Newton's method 31

11 Convexity **32**

12 Constrained Optimization **35**
12.1 Lagrange multipliers 35
12.2 Inequality constraints and the KKT conditions 37

13 Key Takeaways **38**

14 Exercises **39**

15 Programming Exercises **41**

Introduction

Almost every machine learning model is trained by solving an optimization problem. A model has parameters, such as the weights of a linear model or the millions of weights of a neural network, which we collect in a vector $\boldsymbol{\theta}$. A loss function $L(\boldsymbol{\theta})$ measures how badly the model with these parameters fits the training data, and training means finding parameters with a small loss. There are far too many parameters to try out all possible values. Instead, we start with some parameters and improve them step by step, and for this we need to know how the loss changes when we change the parameters slightly. This information is contained in the derivative of L. Vector calculus is the study of derivatives of functions of several variables. In this lecture:

- we extend the derivative to functions $f : \mathbb{R}^n \to \mathbb{R}^m$: partial derivatives, gradients and Jacobian matrices;
- we prove the chain rule, which says that the derivative of a composition of functions is the product of their Jacobian matrices; it is the basis of backpropagation, the algorithm that computes the gradients of neural networks;
- we compute gradients with respect to vectors and matrices, such as the weight matrices of a neural network;
- we use second derivatives (the Hessian matrix) and Taylor's theorem to describe the shape of a function near a point;
- we apply all of this to optimization: critical points, gradient descent, Newton's method, convexity, and optimization with constraints (Lagrange multipliers).

We use the linear algebra of Lectures 1 and 2 throughout: derivatives are linear maps, given by matrices; the direction of steepest ascent comes from the Cauchy–Schwarz inequality; and the shape of a function near a minimum is described by the eigenvalues of a symmetric matrix. Some results from analysis, such as the existence of maxima on closed and bounded sets and the implicit function theorem, are only stated here; the separate appendix on topology and differential topology proves them and puts them in a wider context.

Notation. In this lecture, vectors are written in bold, $\mathbf{x} = (x_1, \dots, x_n)^\top \in \mathbb{R}^n$, to distinguish them from numbers. As before, vectors are columns, $\mathbf{e}_1, \dots, \mathbf{e}_n$ is the standard basis of $\mathbb{R}^n$, $\|\cdot\|$ is the Euclidean norm, and log is the natural logarithm.

1 Multivariable Functions

Definition 1.1. A function $f : \mathbb{R}^n \to \mathbb{R}^m$ is a mapping that takes an input vector $\mathbf{x} \in \mathbb{R}^n$ and produces an output vector $\mathbf{y} = f(\mathbf{x}) \in \mathbb{R}^m$. The components of the output vector are given by $y_i = f_i(\mathbf{x})$ for $i = 1, 2, \ldots, m$, where each $f_i : \mathbb{R}^n \to \mathbb{R}$ is a scalar-valued function.

Example 1.2. $f(x, y) = x^2 + y^2$ is a function from $\mathbb{R}^2$ to $\mathbb{R}$, which takes a point (x, y) in the plane and maps it to the sum of the squares of its coordinates.

Example 1.3 (Functions in machine learning)**.**

1. A linear model with weight vector $\mathbf{w} \in \mathbb{R}^n$ and bias $b \in \mathbb{R}$ is the function $f : \mathbb{R}^n \to \mathbb{R}$, $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$.

2. A layer of a neural network is a function $f : \mathbb{R}^n \to \mathbb{R}^m$, $f(\mathbf{x}) = \sigma(W\mathbf{x} + \mathbf{b})$, where $W \in \mathbb{R}^{m \times n}$ is a weight matrix, $\mathbf{b} \in \mathbb{R}^m$ is a bias vector, and an activation function $\sigma : \mathbb{R} \to \mathbb{R}$ is applied to each entry of $W\mathbf{x} + \mathbf{b}$. Common activation functions are the sigmoid function $\sigma(t) = \frac{1}{1+e^{-t}}$, the hyperbolic tangent tanh, and the rectified linear unit $\mathrm{ReLU}(t) = \max(0, t)$.

3. The softmax function $\mathrm{softmax} : \mathbb{R}^K \to \mathbb{R}^K$,

$$\mathrm{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{e^{z_1} + \cdots + e^{z_K}}, \qquad i = 1, \ldots, K,$$

   turns a vector of scores $\mathbf{z}$ into a vector of probabilities: its entries are positive and sum to 1. Classifiers with K classes use it to predict the probability of each class.

4. Given training data $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)$ with $\mathbf{x}_i \in \mathbb{R}^n$ and $y_i \in \mathbb{R}$, the mean squared error of a linear model without bias,

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} (\mathbf{w}^\top \mathbf{x}_i - y_i)^2,$$

   is a function $L : \mathbb{R}^n \to \mathbb{R}$ of the weights.

The last example shows an important change of perspective. When we use a model, its parameters are fixed and the input $\mathbf{x}$ is the variable. When we train the model, the data are fixed and the parameters are the variables. The derivatives that we need for training are derivatives with respect to the parameters.

Functions are not always defined on all of $\mathbb{R}^n$; for example, $f(\mathbf{x}) = \log(x_1) + x_2$ is only defined when $x_1 > 0$. We then write $f : D \to \mathbb{R}^m$, where $D \subseteq \mathbb{R}^n$ is the domain of f. To take derivatives at a point $\mathbf{a} \in D$, we need to be able to move away from $\mathbf{a}$ a little in every direction without leaving D.

Definition 1.4 (Open sets)**.** The open ball with center $\mathbf{a} \in \mathbb{R}^n$ and radius $r > 0$ is $B(\mathbf{a}, r) = \{\mathbf{x} \in \mathbb{R}^n : \|\mathbf{x} - \mathbf{a}\| < r\}$. A set $D \subseteq \mathbb{R}^n$ is open if for every $\mathbf{a} \in D$, there is an $r > 0$ with $B(\mathbf{a}, r) \subseteq D$.

For example, $\mathbb{R}^n$, every open ball, and the half-plane $\{\mathbf{x} \in \mathbb{R}^2 : x_1 > 0\}$ are open. The closed half-plane $\{\mathbf{x} \in \mathbb{R}^2 : x_1 \geq 0\}$ is not open, since every ball around a point with $x_1 = 0$ contains points with $x_1 < 0$.

1.1 Graphs and level sets

Definition 1.5 (Graph and level sets)**.** Let $f : D \to \mathbb{R}$ with $D \subseteq \mathbb{R}^n$. The graph of f is the set

$$\{(\mathbf{x}, f(\mathbf{x})) : \mathbf{x} \in D\} \subseteq \mathbb{R}^{n+1},$$

and for $c \in \mathbb{R}$, the level set of f at level c is the set $\{\mathbf{x} \in D : f(\mathbf{x}) = c\}$. For $n = 2$, the level sets are usually curves, called level curves or contour lines, and a picture of several level curves is a contour plot.

For $n = 2$, the graph is a surface in $\mathbb{R}^3$, a landscape whose height above the point (x, y) is $f(x, y)$. A contour plot is like a topographic map of this landscape. For $n \geq 3$, we cannot draw the graph, but level sets and contour plots of two-dimensional slices are still useful; for example, contour plots of loss functions are used to visualize the training of neural networks.

Example 1.6. For $f(x, y) = x^2 + y^2$ (Example 1.2), the graph is a bowl-shaped surface, a paraboloid. The level set at level c is the circle of radius $\sqrt{c}$ around the origin if $c > 0$, the single point $(0, 0)$ if $c = 0$, and empty if $c < 0$. The function $g(x, y) = x^2 - y^2$ has a saddle-shaped graph: along the x-axis, it increases away from the origin, and along the y-axis, it decreases. Its level sets are hyperbolas for $c \neq 0$, and the pair of lines $y = \pm x$ for $c = 0$. See Figure 1.

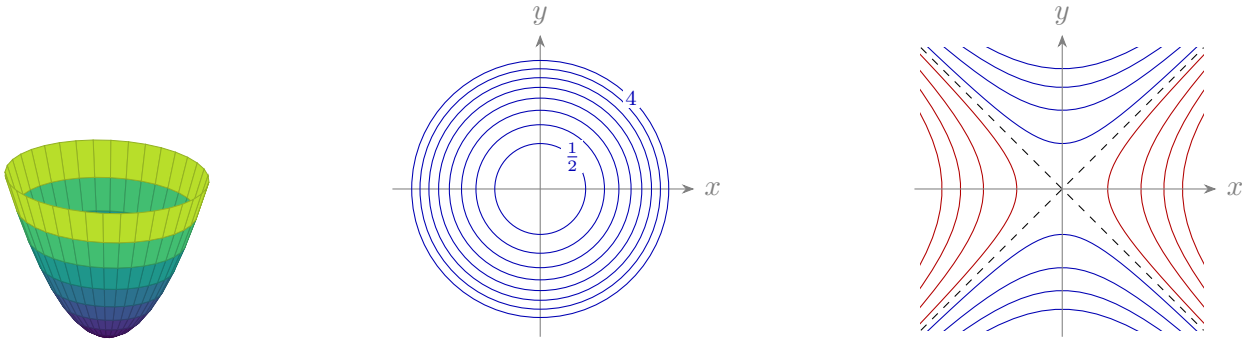


Figure 1: Left: the graph of $f(x, y) = x^2 + y^2$, a paraboloid. Middle: its level curves for the levels $c = \frac{1}{2}, 1, \frac{3}{2}, \ldots, 4$, which are circles of radius $\sqrt{c}$. They get closer together as we move away from the origin, because f gets steeper. Right: level curves of $g(x, y) = x^2 - y^2$ for $c = \pm\frac{1}{2}, \pm\frac{3}{2}, \pm\frac{5}{2}, \pm\frac{7}{2}$ (red: $c > 0$, blue: $c < 0$), and the level set $c = 0$ (dashed).

1.2 Limits and continuity

Definition 1.7 (Limits and continuity)**.** Let $f : D \to \mathbb{R}^m$ with $D \subseteq \mathbb{R}^n$, and let $\mathbf{a} \in \mathbb{R}^n$ be a point such that all points $\mathbf{x} \neq \mathbf{a}$ of some ball $B(\mathbf{a}, r)$ lie in D. We say that $f(\mathbf{x})$ converges to $\mathbf{L} \in \mathbb{R}^m$ as $\mathbf{x}$ tends to $\mathbf{a}$, and write $\lim_{\mathbf{x}\to\mathbf{a}} f(\mathbf{x}) = \mathbf{L}$, if for every $\varepsilon > 0$ there is a $\delta > 0$ such that

$$\|f(\mathbf{x}) - \mathbf{L}\| < \varepsilon \qquad \text{for all } \mathbf{x} \in D \text{ with } 0 < \|\mathbf{x} - \mathbf{a}\| < \delta.$$

If $\mathbf{a} \in D$, then f is continuous at $\mathbf{a}$ if $\lim_{\mathbf{x}\to\mathbf{a}} f(\mathbf{x}) = f(\mathbf{a})$. The function f is continuous if it is continuous at every point of D.

By the inequalities $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1 \leq n\|\mathbf{x}\|_\infty$ of Lecture 2, we could use the ℓ_1 or the ℓ_∞ norm instead of the Euclidean norm in this definition, without changing its meaning. In particular, since

$$|f_i(\mathbf{x}) - L_i| \leq \|f(\mathbf{x}) - \mathbf{L}\| \leq |f_1(\mathbf{x}) - L_1| + \cdots + |f_m(\mathbf{x}) - L_m|,$$

$f(\mathbf{x})$ converges to $\mathbf{L}$ if and only if every component $f_i(\mathbf{x})$ converges to L_i, and f is continuous if and only if all its components are continuous.

As for functions of one variable, sums, products, quotients (where the denominator is not zero) and compositions of continuous functions are continuous, and the coordinate functions $\mathbf{x} \mapsto x_i$ are continuous, since $|x_i - a_i| \leq \|\mathbf{x} - \mathbf{a}\|$. Hence polynomials in $x_1, \ldots, x_n$, linear maps $\mathbf{x} \mapsto A\mathbf{x}$, and functions such as $e^{x_1}\sin(x_2)$ or the functions of Example 1.3 are continuous.

Example 1.8. Let $f(x,y) = \frac{xy}{x^2+y^2}$ for $(x,y) \neq (0,0)$, and $f(0,0) = 0$. On the coordinate axes, f is 0. On the line $y = x$, we have $f(t,t) = \frac{t^2}{2t^2} = \frac{1}{2}$ for $t \neq 0$. So arbitrarily close to the origin, there are points where f is 0 and points where f is $\frac{1}{2}$, and $f(\mathbf{x})$ has no limit as $\mathbf{x} \to \mathbf{0}$. In particular, f is not continuous at the origin, although the functions $t \mapsto f(t,0)$ and $t \mapsto f(0,t)$ are continuous (they are constant). More generally, f is constant on every line through the origin: $f(t,mt) = \frac{m}{1+m^2}$ for $t \neq 0$.

The lesson of this example is that to understand a function near a point, it is not enough to look at it along the coordinate axes. We will meet this function again when we discuss derivatives.

2 Derivatives of Functions of One Variable

We first recall derivatives of functions $f : \mathbb{R} \to \mathbb{R}$, and rewrite the definition in a form that generalizes to several variables.

Definition 2.1 (Derivative)**.** Let $f : D \to \mathbb{R}$ with $D \subseteq \mathbb{R}$ open, and let $x \in D$. The function f is differentiable at x if the limit

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

exists. Then $f'(x)$, also written $\frac{df}{dx}(x)$, is the derivative of f at x. The function f is differentiable if it is differentiable at every point of D.

The quotient $\frac{f(x+h)-f(x)}{h}$ is the slope of the line through the points $(x, f(x))$ and $(x+h, f(x+h))$ of the graph, and its limit $f'(x)$ is the slope of the tangent line to the graph at x.

Proposition 2.2 (Linear approximation)**.** *Let $c \in \mathbb{R}$. Then f is differentiable at x with $f'(x) = c$ if and only if*

$$f(x+h) = f(x) + ch + r(h), \qquad \textit{where } \lim_{h \to 0} \frac{r(h)}{h} = 0.$$

Proof. Let $r(h) = f(x+h) - f(x) - ch$. Then $\frac{r(h)}{h} = \frac{f(x+h)-f(x)}{h} - c$, which converges to 0 if and only if the difference quotient converges to c. □

We write $r(h) = o(|h|)$, read "little o of $|h|$", if $\frac{r(h)}{|h|} \to 0$ as $h \to 0$: the error $r(h)$ is small even compared with the small number $|h|$. So f is differentiable at x if and only if near x, it is approximated by the affine function $h \mapsto f(x) + f'(x)h$, with an error that is $o(|h|)$. This description of the derivative, as the slope of the best linear approximation, is the one that generalizes to functions of several variables.

Proposition 2.3 (Rules of differentiation)**.** *Let f and g be differentiable at x, and let $\lambda \in \mathbb{R}$. Then:*

1. *$(f+g)'(x) = f'(x) + g'(x)$ and $(\lambda f)'(x) = \lambda f'(x)$;*
2. *product rule: $(fg)'(x) = f'(x)g(x) + f(x)g'(x)$;*

3. *quotient rule: if* $g(x) \neq 0$*, then* $\left(\frac{f}{g}\right)'(x) = \frac{f'(x)g(x) - f(x)g'(x)}{g(x)^2}$*;*

4. *chain rule: if* φ *is differentiable at* $g(x)$*, then* $(\varphi \circ g)'(x) = \varphi'(g(x))\, g'(x)$*.*

We assume these rules from calculus, together with the derivatives of the standard functions: $(x^k)' = kx^{k-1}$, $(e^x)' = e^x$, $(\log x)' = \frac{1}{x}$ for $x > 0$, $\sin' = \cos$ and $\cos' = -\sin$. Part (1) says that differentiation is a linear map, as in Lecture 1.

Example 2.4 (Activation functions)**.** The sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$ has the derivative

$$\sigma'(x) = \frac{e^{-x}}{(1+e^{-x})^2} = \sigma(x)(1 - \sigma(x)),$$

since $1 - \sigma(x) = \frac{e^{-x}}{1+e^{-x}}$. So the derivative can be computed from the value $\sigma(x)$, which saves work when training neural networks. Since $t(1-t) \leq \frac{1}{4}$ for all t, we have $0 < \sigma'(x) \leq \frac{1}{4}$, with the maximum at $x = 0$. Similarly, $\tanh'(x) = 1 - \tanh(x)^2$. The ReLU function has derivative 0 for $x < 0$ and 1 for $x > 0$, but it is not differentiable at 0: the difference quotient $\frac{\mathrm{ReLU}(h) - \mathrm{ReLU}(0)}{h}$ is 1 for $h > 0$ and 0 for $h < 0$. In practice, software simply uses the derivative 0 at $x = 0$; since an input is very rarely exactly 0, this causes no problems.

If f' is differentiable, its derivative is the second derivative f'', and similarly we define the k-th derivative $f^{(k)}$. A function is k times continuously differentiable if $f^{(k)}$ exists and is continuous.

2.1 Taylor polynomials

The linear approximation of f near x_0 only uses $f(x_0)$ and $f'(x_0)$. Using higher derivatives, we get better approximations by polynomials.

Definition 2.5 (Taylor polynomial)**.** Let f be k times differentiable at x_0. The Taylor polynomial of degree k of f at x_0 is

$$\begin{aligned} T_k(x) &= \sum_{j=0}^{k} \frac{f^{(j)}(x_0)}{j!}(x - x_0)^j \\ &= f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2}(x - x_0)^2 + \cdots + \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k. \end{aligned}$$

The coefficients are chosen such that T_k and f have the same derivatives of orders $0, 1, \ldots, k$ at x_0: the j-th derivative of $\frac{(x-x_0)^i}{i!}$ at x_0 is 1 if $i = j$, and 0 otherwise. The Taylor polynomial $T_1(x) = f(x_0) + f'(x_0)(x - x_0)$ is the tangent line.

Theorem 2.6 (Taylor's theorem)**.** *Let* f *be* $k+1$ *times differentiable on an open interval* I *that contains* x_0 *and* x*. Then there is a number* ξ *between* x_0 *and* x *such that*

$$f(x) = T_k(x) + \frac{f^{(k+1)}(\xi)}{(k+1)!}(x - x_0)^{k+1}.$$

For $k = 0$, this is the mean value theorem $f(x) - f(x_0) = f'(\xi)(x - x_0)$. We do not prove the theorem here. It says that if $|f^{(k+1)}| \leq M$ on I, then the error of the approximation $f(x) \approx T_k(x)$ is at most $\frac{M}{(k+1)!}|x - x_0|^{k+1}$, which is tiny when x is close to x_0.

Example 2.7. All derivatives of e^x are e^x, so its Taylor polynomials at $x_0 = 0$ are $T_k(x) = 1 + x + \frac{x^2}{2!} + \cdots + \frac{x^k}{k!}$. For example, $e^{0.1} \approx T_2(0.1) = 1 + 0.1 + 0.005 = 1.105$. By Taylor's theorem, the error is $\frac{e^\xi}{3!}(0.1)^3$ for some $0 < \xi < 0.1$, which is at most $\frac{e^{0.1}}{6000} < 0.0002$. Indeed, $e^{0.1} = 1.10517\ldots$.

Example 2.8. The derivatives of cos at 0 are $1, 0, -1, 0, 1, 0, -1, 0, \ldots$, so the Taylor polynomials of cos at 0 are

$$T_{2k}(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \cdots + (-1)^k \frac{x^{2k}}{(2k)!}.$$

Figure 2 shows that they approximate cos on larger and larger intervals. Since all derivatives of cos are bounded by 1, Taylor's theorem gives $|\cos x - T_k(x)| \le \frac{|x|^{k+1}}{(k+1)!}$, which tends to 0 as $k \to \infty$ for every fixed x, since factorials grow faster than powers. Hence $\cos x = \sum_{j=0}^{\infty} (-1)^j \frac{x^{2j}}{(2j)!}$ for all $x \in \mathbb{R}$: the Taylor series of cos converges to cos everywhere. Not all functions behave this nicely. For example, the Taylor series $x - \frac{x^2}{2} + \frac{x^3}{3} - \cdots$ of $\log(1+x)$ at 0 only converges for $-1 < x \le 1$.

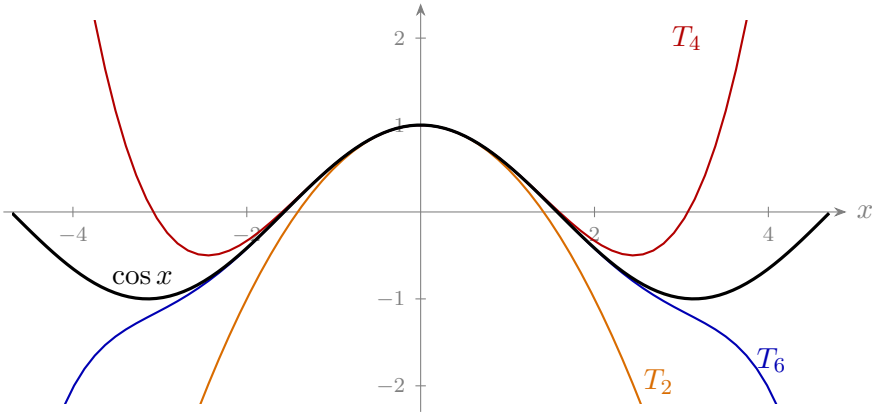


Figure 2: The function $\cos x$ (black) and its Taylor polynomials T_2, T_4 and T_6 at 0. Near 0, all of them are good approximations, and higher degrees give good approximations on larger intervals.

Taylor's theorem also tells us how to compute derivatives numerically, and how accurate the result is.

Proposition 2.9 (Finite differences)**.** *Let $h > 0$, and let f be three times differentiable on an open interval containing $[x-h, x+h]$, with $|f''| \le M_2$ and $|f'''| \le M_3$ there. Then*

$$\left|\frac{f(x+h) - f(x)}{h} - f'(x)\right| \le \frac{M_2}{2}h \qquad \text{and} \qquad \left|\frac{f(x+h) - f(x-h)}{2h} - f'(x)\right| \le \frac{M_3}{6}h^2.$$

Proof. By Taylor's theorem with $k = 1$, $f(x+h) = f(x) + f'(x)h + \frac{f''(\xi)}{2}h^2$, so the error of the forward difference is $\frac{f''(\xi)}{2}h$. With $k = 2$,

$$f(x \pm h) = f(x) \pm f'(x)h + \frac{f''(x)}{2}h^2 \pm \frac{f'''(\xi_\pm)}{6}h^3$$

for some ξ_+ and ξ_-. Subtracting the two equations gives $f(x+h) - f(x-h) = 2f'(x)h + \frac{f'''(\xi_+) + f'''(\xi_-)}{6}h^3$. We divide by $2h$; the error is $\frac{f'''(\xi_+) + f'''(\xi_-)}{12}h^2$, which is at most $\frac{M_3}{6}h^2$. □

The central difference $\frac{f(x+h)-f(x-h)}{2h}$ is much more accurate than the forward difference: halving h divides its error by 4 instead of 2. In machine learning, it is used for gradient checking, i.e., to test a program that computes derivatives. The step h cannot be chosen arbitrarily small, though: computers store numbers with about 16 significant digits, and in the difference $f(x+h) - f(x-h)$ of two almost equal numbers, most of these digits cancel (see Programming Exercise P1).

3 Partial Derivatives and Gradients

For a function of several variables, the simplest way to take a derivative is to vary only one variable and to keep all others fixed.

Definition 3.1 (Partial derivatives and gradient)**.** Let $f : D \to \mathbb{R}$ with $D \subseteq \mathbb{R}^n$ open, and let $\mathbf{x} \in D$. The partial derivative of f with respect to x_i at $\mathbf{x}$ is

$$\frac{\partial f}{\partial x_i}(\mathbf{x}) = \lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h},$$

if this limit exists. If all partial derivatives of f at $\mathbf{x}$ exist, the gradient of f at $\mathbf{x}$ is the column vector

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{pmatrix} \in \mathbb{R}^n.$$

The partial derivative $\frac{\partial f}{\partial x_i}(\mathbf{x})$ is the ordinary derivative at $t = x_i$ of the function of one variable

$$t \mapsto f(x_1, \ldots, x_{i-1}, t, x_{i+1}, \ldots, x_n).$$

So we compute it with the rules of Section 2, treating all variables other than x_i as constants.

Example 3.2. Let $f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3$. Treating x_2 as a constant, and then x_1 as a constant, we get

$$\frac{\partial f}{\partial x_1} = 2x_1 x_2 + x_2^3, \qquad \frac{\partial f}{\partial x_2} = x_1^2 + 3x_1 x_2^2.$$

At $\mathbf{x} = (1, 2)^\top$, the gradient is $\nabla f(\mathbf{x}) = (4 + 8,\, 1 + 12)^\top = (12, 13)^\top$.

Proposition 3.3. *Let $\mathbf{a} \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. Then for all $\mathbf{x} \in \mathbb{R}^n$:*

1. $\nabla(\mathbf{a}^\top \mathbf{x}) = \mathbf{a}$;
2. $\nabla(\mathbf{x}^\top A\mathbf{x}) = (A + A^\top)\mathbf{x}$; *in particular,* $\nabla(\mathbf{x}^\top A\mathbf{x}) = 2A\mathbf{x}$ *if A is symmetric, and* $\nabla\|\mathbf{x}\|^2 = 2\mathbf{x}$;
3. $\nabla\|\mathbf{x}\| = \frac{\mathbf{x}}{\|\mathbf{x}\|}$ *for* $\mathbf{x} \neq \mathbf{0}$.

Proof. (1) Since $\mathbf{a}^\top \mathbf{x} = \sum_j a_j x_j$, the partial derivative with respect to x_k is a_k.

(2) We have $\mathbf{x}^\top A\mathbf{x} = \sum_{i,j} A_{ij} x_i x_j$. The terms that contain x_k are those with $i = k$ or $j = k$, and differentiating them gives

$$\frac{\partial}{\partial x_k} \mathbf{x}^\top A\mathbf{x} = \sum_j A_{kj} x_j + \sum_i A_{ik} x_i = (A\mathbf{x})_k + (A^\top \mathbf{x})_k.$$

(The term $A_{kk} x_k^2$ has the derivative $2A_{kk} x_k$, which is counted once in each sum.) For $A = I_n$, we get $\nabla\|\mathbf{x}\|^2 = \nabla(\mathbf{x}^\top \mathbf{x}) = 2\mathbf{x}$.

(3) By the chain rule, $\frac{\partial}{\partial x_k}(x_1^2 + \cdots + x_n^2)^{1/2} = \frac{1}{2}(x_1^2 + \cdots + x_n^2)^{-1/2} \cdot 2x_k = \frac{x_k}{\|\mathbf{x}\|}$. □

These formulas look like their one-variable versions $(ax)' = a$ and $(ax^2)' = 2ax$, and they will be used constantly. Note that the gradient of the Euclidean norm is a unit vector, which points away from the origin.

3.1 Differentiability

Partial derivatives only describe f along the coordinate axes, and by Example 1.8, this is not enough. The function $f(x, y) = \frac{xy}{x^2+y^2}$ of that example is 0 on both axes, so both partial derivatives at the origin exist and are 0. But f is not even continuous at the origin, so it would be strange to call it differentiable there. The right definition generalizes Proposition 2.2: f should be approximated by a linear function near $\mathbf{x}$, in all directions at once. As before, we write $r(\mathbf{h}) = o(\|\mathbf{h}\|)$ if $\frac{r(\mathbf{h})}{\|\mathbf{h}\|} \to 0$ as $\mathbf{h} \to \mathbf{0}$.

Definition 3.4 (Differentiable function)**.** Let $f : D \to \mathbb{R}$ with $D \subseteq \mathbb{R}^n$ open, and let $\mathbf{x} \in D$. The function f is differentiable at $\mathbf{x}$ if there is a row vector $\mathbf{c} \in \mathbb{R}^{1\times n}$ such that

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \mathbf{ch} + r(\mathbf{h}), \qquad \text{where } r(\mathbf{h}) = o(\|\mathbf{h}\|).$$

The row vector $\mathbf{c}$, or the linear map $\mathbf{h} \mapsto \mathbf{ch}$, is then called the derivative of f at $\mathbf{x}$, and is denoted by $Df(\mathbf{x})$. The function f is differentiable if it is differentiable at every point of D.

Proposition 3.5. *Let f be differentiable at $\mathbf{x}$, with derivative $Df(\mathbf{x}) = \mathbf{c}$. Then:*

1. *for every vector $\mathbf{v} \in \mathbb{R}^n$, the directional derivative of f at $\mathbf{x}$ in the direction $\mathbf{v}$,*

$$D_{\mathbf{v}} f(\mathbf{x}) = \lim_{t\to 0} \frac{f(\mathbf{x} + t\mathbf{v}) - f(\mathbf{x})}{t},$$

   exists and is equal to $\mathbf{cv}$;

2. *all partial derivatives of f at $\mathbf{x}$ exist, and $\mathbf{c} = (\frac{\partial f}{\partial x_1}(\mathbf{x}), \ldots, \frac{\partial f}{\partial x_n}(\mathbf{x}))$. In other words, $Df(\mathbf{x}) = \nabla f(\mathbf{x})^\top$, and $D_{\mathbf{v}} f(\mathbf{x}) = \nabla f(\mathbf{x})^\top \mathbf{v}$;*

3. *f is continuous at $\mathbf{x}$.*

Proof. (1) For $\mathbf{v} = \mathbf{0}$, both sides are 0. For $\mathbf{v} \neq \mathbf{0}$ and $t \neq 0$, we apply the definition with $\mathbf{h} = t\mathbf{v}$:

$$\left| \frac{f(\mathbf{x} + t\mathbf{v}) - f(\mathbf{x})}{t} - \mathbf{cv} \right| = \frac{|r(t\mathbf{v})|}{|t|} = \|\mathbf{v}\| \frac{|r(t\mathbf{v})|}{\|t\mathbf{v}\|},$$

which tends to 0 as $t \to 0$.

(2) The partial derivative $\frac{\partial f}{\partial x_i}(\mathbf{x})$ is the directional derivative in the direction $\mathbf{e}_i$, which by (1) is $\mathbf{ce}_i = c_i$.

(3) We have $|f(\mathbf{x} + \mathbf{h}) - f(\mathbf{x})| \leq |\mathbf{ch}| + |r(\mathbf{h})| \leq \|\mathbf{c}\|\|\mathbf{h}\| + |r(\mathbf{h})|$ by the Cauchy–Schwarz inequality, and both terms tend to 0 as $\mathbf{h} \to \mathbf{0}$. □

By (3), the function of Example 1.8 is not differentiable at the origin, although its partial derivatives exist there. So differentiability is a stronger condition than the existence of partial derivatives. Fortunately, the following theorem, which we do not prove, shows that almost all functions that we meet in practice are differentiable.

Theorem 3.6. *Let $f : D \to \mathbb{R}$ with $D \subseteq \mathbb{R}^n$ open. If all partial derivatives of f exist in a ball around $\mathbf{x}$ and are continuous at $\mathbf{x}$, then f is differentiable at $\mathbf{x}$.*

A function whose partial derivatives exist and are continuous on D is called continuously differentiable. By Theorem 3.6, polynomials, and functions built from exp, log, sin, the sigmoid function, etc., by sums, products, quotients and compositions, are differentiable wherever they are defined. Some important functions in machine learning are differentiable only at most points, such as ReLU, the ℓ_1 norm, and the hinge loss $\max(0, 1-t)$; at the few points where they are not differentiable, software uses a reasonable substitute for the derivative.

Remark 3.7 (Notation). In this lecture, as in the remark on gradients in the section on dual spaces in Lecture 1, the derivative $Df(\mathbf{x})$ of a function $f : \mathbb{R}^n \to \mathbb{R}$ is a row vector, and the gradient $\nabla f(\mathbf{x}) = Df(\mathbf{x})^\top$ is a column vector. Some books call the row vector the gradient. In machine learning software such as PyTorch, the gradient with respect to a parameter always has the same shape as the parameter, so it is a column vector for a vector of parameters, and a matrix for a matrix of parameters (Section 6).

Remark 3.8 (The gradient depends on the inner product). The derivative $Df(\mathbf{x})$ is a linear functional $\mathbb{R}^n \to \mathbb{R}$, i.e., an element of the dual space $(\mathbb{R}^n)^*$ of Lecture 1. The gradient is the vector that represents this linear functional with respect to the dot product:

$$Df(\mathbf{x})\mathbf{h} = \langle \nabla f(\mathbf{x}), \mathbf{h} \rangle \qquad \text{for all } \mathbf{h} \in \mathbb{R}^n.$$

In every finite-dimensional inner product space, every linear functional is represented in this way by a unique vector; this is the Riesz representation theorem. If we use a different inner product on $\mathbb{R}^n$, such as $\langle \mathbf{x}, \mathbf{y} \rangle_A = \mathbf{x}^\top A \mathbf{y}$ for a symmetric positive definite matrix A (Lecture 2), then the vector that represents $Df(\mathbf{x})$ changes: since

$$\langle A^{-1}\nabla f(\mathbf{x}), \mathbf{h} \rangle_A = \nabla f(\mathbf{x})^\top A^{-1} A \mathbf{h} = Df(\mathbf{x})\mathbf{h},$$

the gradient with respect to $\langle \cdot, \cdot \rangle_A$ is $A^{-1}\nabla f(\mathbf{x})$. So the derivative is determined by f alone, but the gradient also depends on how we measure lengths and angles. Changing the inner product is the idea behind preconditioning and the natural gradient method, and Newton's method in Section 10.3 is an example.

3.2 Directional derivatives and steepest ascent

By Proposition 3.5, the rate of change of a differentiable function in the direction $\mathbf{v}$ is $D_{\mathbf{v}} f(\mathbf{x}) = \nabla f(\mathbf{x})^\top \mathbf{v}$. In which direction does f increase fastest?

Theorem 3.9 (Steepest ascent)**.** *Let f be differentiable at $\mathbf{x}$, with $\nabla f(\mathbf{x}) \neq \mathbf{0}$. Then for every unit vector $\mathbf{v}$,*

$$-\|\nabla f(\mathbf{x})\| \leq D_{\mathbf{v}} f(\mathbf{x}) \leq \|\nabla f(\mathbf{x})\|.$$

The maximum is attained only for $\mathbf{v} = \frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|}$, and the minimum only for $\mathbf{v} = -\frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|}$.

Proof. By the Cauchy–Schwarz inequality (Lecture 2), $|D_{\mathbf{v}} f(\mathbf{x})| = |\nabla f(\mathbf{x})^\top \mathbf{v}| \leq \|\nabla f(\mathbf{x})\| \|\mathbf{v}\| = \|\nabla f(\mathbf{x})\|$, with equality if and only if $\mathbf{v}$ is a multiple of $\nabla f(\mathbf{x})$. The only unit vectors that are multiples of $\nabla f(\mathbf{x})$ are $\pm \frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|}$, and they give $D_{\mathbf{v}} f(\mathbf{x}) = \pm \|\nabla f(\mathbf{x})\|$. □

So the gradient points in the direction in which f increases fastest, and its length is the rate of increase in this direction. The negative gradient $-\nabla f(\mathbf{x})$ is the direction of steepest descent, which is the direction that gradient descent uses to decrease a loss function (Section 10). In directions orthogonal to $\nabla f(\mathbf{x})$, the rate of change is 0; we will see in Section 5 that these are the directions tangent to the level set through $\mathbf{x}$.

Example 3.10. For the function $f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3$ of Example 3.2, at $\mathbf{x} = (1, 2)^\top$, we found $\nabla f(\mathbf{x}) = (12, 13)^\top$. In the direction of the unit vector $\mathbf{v} = \frac{1}{5}(3, 4)^\top$, the rate of change is $D_{\mathbf{v}} f(\mathbf{x}) = \frac{1}{5}(36+52) = 17.6$. The largest possible rate is $\|\nabla f(\mathbf{x})\| = \sqrt{313} \approx 17.69$, in the direction $\frac{1}{\sqrt{313}}(12, 13)^\top$, and in the orthogonal direction $\frac{1}{\sqrt{313}}(13, -12)^\top$ the rate of change is 0.

4 Jacobians: Derivatives of Vector-Valued Functions

Many functions in machine learning have vector outputs, such as the layers of a neural network. Their derivatives are matrices.

Definition 4.1 (Jacobian)**.** Let $f : D \to \mathbb{R}^m$ with $D \subseteq \mathbb{R}^n$ open, and let $\mathbf{x} \in D$. The function f is differentiable at $\mathbf{x}$ if there is an $m \times n$ matrix J such that

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + J\mathbf{h} + \mathbf{r}(\mathbf{h}), \qquad \text{where } \lim_{\mathbf{h} \to \mathbf{0}} \frac{\|\mathbf{r}(\mathbf{h})\|}{\|\mathbf{h}\|} = 0.$$

The matrix J is called the Jacobian matrix, or the derivative, of f at $\mathbf{x}$, and is denoted by $Df(\mathbf{x})$ or $J_f(\mathbf{x})$.

For $m = 1$, this is Definition 3.4. The next proposition shows that the Jacobian consists of the derivatives of the components of f.

Proposition 4.2. *A function $f = (f_1, \ldots, f_m)^\top$ is differentiable at $\mathbf{x}$ if and only if all its components f_i are differentiable at $\mathbf{x}$. In this case, the i-th row of $Df(\mathbf{x})$ is $Df_i(\mathbf{x}) = \nabla f_i(\mathbf{x})^\top$, i.e.,*

$$Df(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial f_1}{\partial x_n}(\mathbf{x}) \\ \vdots & & \vdots \\ \frac{\partial f_m}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial f_m}{\partial x_n}(\mathbf{x}) \end{pmatrix}, \qquad \textit{that is,} \quad \big(Df(\mathbf{x})\big)_{ij} = \frac{\partial f_i}{\partial x_j}(\mathbf{x}).$$

Proof. The i-th component of the equation in Definition 4.1 is $f_i(\mathbf{x} + \mathbf{h}) = f_i(\mathbf{x}) + J_i\mathbf{h} + r_i(\mathbf{h})$, where J_i is the i-th row of J. Since $|r_i(\mathbf{h})| \le \|\mathbf{r}(\mathbf{h})\| \le |r_1(\mathbf{h})| + \cdots + |r_m(\mathbf{h})|$, we have $\frac{\|\mathbf{r}(\mathbf{h})\|}{\|\mathbf{h}\|} \to 0$ if and only if $\frac{r_i(\mathbf{h})}{\|\mathbf{h}\|} \to 0$ for all i. So f is differentiable with Jacobian J if and only if every f_i is differentiable with derivative J_i, and by Proposition 3.5, $J_i = \nabla f_i(\mathbf{x})^\top$. □

So the rows of the Jacobian correspond to the outputs of f, and the columns to the inputs: the j-th column $\frac{\partial f}{\partial x_j}(\mathbf{x})$ describes how the output vector changes when we change x_j. In the language of Lecture 1, the derivative of f at $\mathbf{x}$ is the linear transformation $\mathbf{h} \mapsto Df(\mathbf{x})\mathbf{h}$ from $\mathbb{R}^n$ to $\mathbb{R}^m$ that best approximates the change $f(\mathbf{x} + \mathbf{h}) - f(\mathbf{x})$, and the Jacobian is its matrix with respect to the standard bases. Near $\mathbf{x}$,

$$f(\mathbf{x} + \mathbf{h}) \approx f(\mathbf{x}) + Df(\mathbf{x})\mathbf{h} :$$

a differentiable function looks like an affine function if we zoom in far enough.

Example 4.3. Let $f(x_1, x_2) = (x_1^2 x_2,\ 3x_1 + x_2^2)^\top$. Then

$$Df(\mathbf{x}) = \begin{pmatrix} 2x_1x_2 & x_1^2 \\ 3 & 2x_2 \end{pmatrix}, \qquad \text{and at } \mathbf{a} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}: \quad f(\mathbf{a}) = \begin{pmatrix} 2 \\ 7 \end{pmatrix}, \quad Df(\mathbf{a}) = \begin{pmatrix} 4 & 1 \\ 3 & 4 \end{pmatrix}.$$

The linear approximation gives $f(1.1, 1.9) \approx f(\mathbf{a}) + Df(\mathbf{a}) \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} = \begin{pmatrix} 2.3 \\ 6.9 \end{pmatrix}$, while the exact value is $(2.299,\ 6.91)^\top$.

Example 4.4 (Affine maps)**.** Let $f(\mathbf{x}) = A\mathbf{x} + \mathbf{b}$ with $A \in \mathbb{R}^{m\times n}$ and $\mathbf{b} \in \mathbb{R}^m$. Then $f(\mathbf{x}+\mathbf{h}) = f(\mathbf{x}) + A\mathbf{h}$ exactly, so $Df(\mathbf{x}) = A$ for every $\mathbf{x}$: the best linear approximation of an affine map is its linear part. For $m = 1$ and $f(\mathbf{x}) = \mathbf{a}^\top\mathbf{x}$, this is Proposition 3.3(1) again.

Example 4.5 (Functions applied entrywise)**.** Let $\varphi : \mathbb{R} \to \mathbb{R}$ be differentiable, such as an activation function, and let $f : \mathbb{R}^n \to \mathbb{R}^n$ apply φ to each entry, $f(\mathbf{x}) = (\varphi(x_1), \dots, \varphi(x_n))^\top$. Since f_i only depends on x_i, we have $\frac{\partial f_i}{\partial x_j} = 0$ for $i \neq j$, so

$$Df(\mathbf{x}) = \operatorname{diag}\left(\varphi'(x_1), \dots, \varphi'(x_n)\right).$$

Multiplying a vector by a diagonal matrix multiplies its entries: $Df(\mathbf{x})\mathbf{v} = \varphi'(\mathbf{x}) \odot \mathbf{v}$, where $\varphi'(\mathbf{x}) = (\varphi'(x_1), \dots, \varphi'(x_n))^\top$, and $\odot$ is the entrywise product (or Hadamard product) $(\mathbf{a} \odot \mathbf{b})_i = a_i b_i$. Computing the product in this way costs n multiplications instead of n^2.

Example 4.6 (Softmax)**.** Let $\mathbf{s} = \operatorname{softmax}(\mathbf{z})$ with $\mathbf{z} \in \mathbb{R}^K$, i.e., $s_i = \frac{e^{z_i}}{S}$ with $S = e^{z_1} + \cdots + e^{z_K}$. Since $\frac{\partial S}{\partial z_j} = e^{z_j}$, the quotient rule gives

$$\frac{\partial s_i}{\partial z_j} = \frac{\delta_{ij} e^{z_i} S - e^{z_i} e^{z_j}}{S^2} = s_i\delta_{ij} - s_i s_j, \qquad \text{where } \delta_{ij} = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{if } i \neq j. \end{cases}$$

So $D\operatorname{softmax}(\mathbf{z}) = \operatorname{diag}(\mathbf{s}) - \mathbf{s}\mathbf{s}^\top$. For example, for $\mathbf{z} = (2, 1, 0)^\top$, we have $\mathbf{s} \approx (0.665, 0.245, 0.090)^\top$ and

$$D\operatorname{softmax}(\mathbf{z}) \approx \begin{pmatrix} 0.223 & -0.163 & -0.060 \\ -0.163 & 0.185 & -0.022 \\ -0.060 & -0.022 & 0.082 \end{pmatrix}.$$

The Jacobian is symmetric, and its rows sum to 0: with $\mathbf{1} = (1, \dots, 1)^\top$, we have $D\operatorname{softmax}(\mathbf{z})\mathbf{1} = \mathbf{s} - \mathbf{s}(\mathbf{s}^\top\mathbf{1}) = \mathbf{0}$, since the entries of $\mathbf{s}$ sum to 1. This reflects a fact from the section on quotient spaces in Lecture 1: $\operatorname{softmax}(\mathbf{z} + c\mathbf{1}) = \operatorname{softmax}(\mathbf{z})$ for all $c \in \mathbb{R}$, so moving $\mathbf{z}$ in the direction $\mathbf{1}$ does not change the output, and $\mathbf{1}$ lies in the null space of the Jacobian.

4.1 The Jacobian determinant

If $m = n$, the Jacobian is a square matrix, and its determinant describes how f changes areas and volumes near a point. By Lecture 1, the linear map $\mathbf{h} \mapsto A\mathbf{h}$ multiplies areas (for $n = 2$) and volumes (for $n \geq 3$) by $|\det A|$. Near $\mathbf{x}$, f is approximately the affine map $\mathbf{h} \mapsto f(\mathbf{x}) + Df(\mathbf{x})\mathbf{h}$. So a small square with side length ε at $\mathbf{x}$ is mapped approximately to the parallelogram spanned by $\varepsilon Df(\mathbf{x})\mathbf{e}_1$ and $\varepsilon Df(\mathbf{x})\mathbf{e}_2$, whose area is $|\det Df(\mathbf{x})|\,\varepsilon^2$: the number $|\det Df(\mathbf{x})|$ is the factor by which f scales small areas near $\mathbf{x}$.

Example 4.7 (Polar coordinates)**.** The map $f(r, \varphi) = (r\cos\varphi,\ r\sin\varphi)^\top$ turns polar coordinates into Cartesian coordinates. Its Jacobian and Jacobian determinant are

$$Df(r, \varphi) = \begin{pmatrix} \cos\varphi & -r\sin\varphi \\ \sin\varphi & r\cos\varphi \end{pmatrix}, \qquad \det Df(r, \varphi) = r\cos^2\varphi + r\sin^2\varphi = r.$$

A small rectangle $[r, r+\Delta r] \times [\varphi, \varphi+\Delta\varphi]$ is mapped to a piece of an annulus, which is approximately a rectangle with sides Δr and $r\Delta\varphi$ (an arc of a circle of radius r), with area approximately $r\,\Delta r\,\Delta\varphi$; see Figure 3. This explains the factor r in the substitution rule for integrals in polar coordinates,

$$\iint_{\mathbb{R}^2} g(x, y)\,dx\,dy = \int_0^{2\pi}\int_0^{\infty} g(r\cos\varphi, r\sin\varphi)\,r\,dr\,d\varphi.$$

A famous application is the Gaussian integral. Since $e^{-(x^2+y^2)} = e^{-x^2}e^{-y^2}$,

$$\left(\int_{-\infty}^{\infty} e^{-x^2}\,dx\right)^2 = \iint_{\mathbb{R}^2} e^{-(x^2+y^2)}\,dx\,dy = \int_0^{2\pi}\int_0^{\infty} e^{-r^2} r\,dr\,d\varphi = 2\pi\left[-\tfrac{1}{2}e^{-r^2}\right]_0^{\infty} = \pi,$$

so $\int_{-\infty}^{\infty} e^{-x^2}dx = \sqrt{\pi}$. This is where the factor $\frac{1}{\sqrt{2\pi}}$ in the density of the normal distribution comes from.

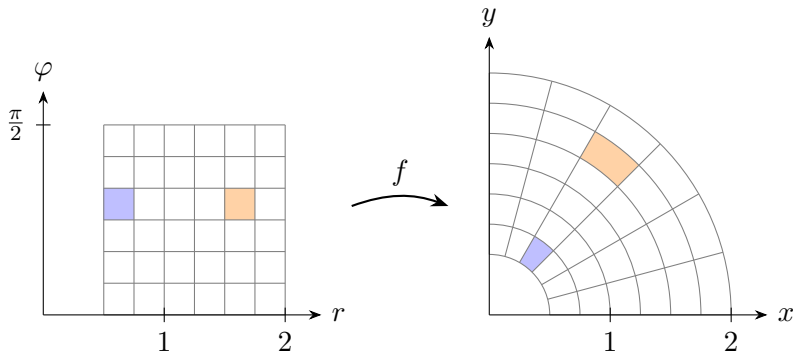


Figure 3: Polar coordinates $f(r,\varphi) = (r\cos\varphi, r\sin\varphi)$. The grid of rectangles on the left is mapped to the grid of annulus pieces on the right. The two shaded rectangles have the same area, but their images do not: since $\det Df(r,\varphi) = r$, areas are stretched more far from the origin. (The image of $[r_1,r_2]\times[\varphi_1,\varphi_2]$ has the area $\frac{r_1+r_2}{2}(r_2-r_1)(\varphi_2-\varphi_1)$, so the orange image is $\frac{1.625}{0.625} = 2.6$ times as large as the blue one.)

Remark 4.8 (Change of variables). More generally, if f is a differentiable bijection between open subsets U and V of $\mathbb{R}^n$ with a differentiable inverse, then $\int_V g(\mathbf{y})\,d\mathbf{y} = \int_U g(f(\mathbf{x}))\,|\det Df(\mathbf{x})|\,d\mathbf{x}$; we do not prove this here. In probability, it says that if a random vector X has the density p_X, then $Y = f(X)$ has the density p_Y with $p_Y(f(\mathbf{x})) = \frac{p_X(\mathbf{x})}{|\det Df(\mathbf{x})|}$. Normalizing flows are generative models that turn simple random vectors (e.g., Gaussian ones) into complicated ones in this way. They use neural networks f whose Jacobian determinant is cheap to compute, for example networks with triangular Jacobians, whose determinant is the product of the diagonal entries (Lecture 1).

5 The Chain Rule

Most functions in machine learning are compositions of simpler functions: a neural network is a composition of layers, and its loss is a composition of the network with a loss function. The chain rule tells us how to differentiate compositions.

Theorem 5.1 (Chain rule)**.** *Let $g: D \to \mathbb{R}^m$ with $D \subseteq \mathbb{R}^n$ open, and $f: E \to \mathbb{R}^k$ with $E \subseteq \mathbb{R}^m$ open and $g(D) \subseteq E$. If g is differentiable at $\mathbf{x}$ and f is differentiable at $g(\mathbf{x})$, then $f\circ g$ is differentiable at $\mathbf{x}$, and*

$$D(f\circ g)(\mathbf{x}) = Df(g(\mathbf{x}))\,Dg(\mathbf{x}).$$

The sizes of the matrices are $(k\times n) = (k\times m)(m\times n)$. In terms of the entries,

$$\frac{\partial(f\circ g)_i}{\partial x_j}(\mathbf{x}) = \sum_{l=1}^{m} \frac{\partial f_i}{\partial y_l}(g(\mathbf{x}))\,\frac{\partial g_l}{\partial x_j}(\mathbf{x}),$$

where $y_1,\dots,y_m$ denote the variables of f.

Proof. Let $\mathbf{y} = g(\mathbf{x})$, $A = Dg(\mathbf{x})$ and $B = Df(\mathbf{y})$. By assumption,

$$g(\mathbf{x}+\mathbf{h}) = \mathbf{y} + \mathbf{k}(\mathbf{h}) \quad \text{with } \mathbf{k}(\mathbf{h}) = A\mathbf{h} + \mathbf{r}(\mathbf{h}), \qquad f(\mathbf{y}+\mathbf{k}) = f(\mathbf{y}) + B\mathbf{k} + \mathbf{s}(\mathbf{k}),$$

where $\frac{\|\mathbf{r}(\mathbf{h})\|}{\|\mathbf{h}\|} \to 0$ as $\mathbf{h} \to \mathbf{0}$ and $\frac{\|\mathbf{s}(\mathbf{k})\|}{\|\mathbf{k}\|} \to 0$ as $\mathbf{k} \to \mathbf{0}$, with $\mathbf{s}(\mathbf{0}) = \mathbf{0}$. Hence

$$f(g(\mathbf{x}+\mathbf{h})) = f(\mathbf{y}) + BA\mathbf{h} + \big(B\mathbf{r}(\mathbf{h}) + \mathbf{s}(\mathbf{k}(\mathbf{h}))\big),$$

and we have to show that the error term in parentheses is $o(\|\mathbf{h}\|)$. We use the spectral norm of Lecture 2, which satisfies $\|M\mathbf{v}\| \le \|M\|_2\|\mathbf{v}\|$. First, $\|B\mathbf{r}(\mathbf{h})\| \le \|B\|_2\|\mathbf{r}(\mathbf{h})\|$, which is $o(\|\mathbf{h}\|)$. Second, for small $\mathbf{h}$, we have $\|\mathbf{r}(\mathbf{h})\| \le \|\mathbf{h}\|$ and therefore $\|\mathbf{k}(\mathbf{h})\| \le (\|A\|_2 + 1)\|\mathbf{h}\|$. Let $\varepsilon > 0$. For small $\mathbf{k}$, we have $\|\mathbf{s}(\mathbf{k})\| \le \varepsilon\|\mathbf{k}\|$, so for small $\mathbf{h}$,

$$\|\mathbf{s}(\mathbf{k}(\mathbf{h}))\| \le \varepsilon\|\mathbf{k}(\mathbf{h})\| \le \varepsilon(\|A\|_2 + 1)\|\mathbf{h}\|.$$

Since ε was arbitrary, $\mathbf{s}(\mathbf{k}(\mathbf{h})) = o(\|\mathbf{h}\|)$ as well. □

The chain rule says that the linear approximation of $f \circ g$ is the composition of the linear approximations of f and g. Since the matrix of a composition of linear maps is the product of their matrices (Lecture 1), the Jacobian of $f \circ g$ is the product of the Jacobians. For $n = m = k = 1$, it is the familiar rule $(f \circ g)'(x) = f'(g(x))g'(x)$. Two special cases are particularly important.

Corollary 5.2. *1. (Functions along curves.) Let $\mathbf{x} : I \to \mathbb{R}^n$, $t \mapsto \mathbf{x}(t) = (x_1(t), \dots, x_n(t))^\top$, be differentiable on an open interval I, and let $f : \mathbb{R}^n \to \mathbb{R}$ be differentiable. Then*

$$\frac{d}{dt} f(\mathbf{x}(t)) = \nabla f(\mathbf{x}(t))^\top \mathbf{x}'(t) = \sum_{i=1}^{n} \frac{\partial f}{\partial x_i}(\mathbf{x}(t))\, x_i'(t), \qquad \text{where } \mathbf{x}'(t) = (x_1'(t), \dots, x_n'(t))^\top.$$

2. (Gradients of compositions.) Let $g : \mathbb{R}^n \to \mathbb{R}^m$ and $f : \mathbb{R}^m \to \mathbb{R}$ be differentiable. Then

$$\nabla(f \circ g)(\mathbf{x}) = Dg(\mathbf{x})^\top \nabla f(g(\mathbf{x})).$$

Proof. (1) The Jacobian of $t \mapsto \mathbf{x}(t)$ is the $n \times 1$ matrix $\mathbf{x}'(t)$, so this is the chain rule with $Df(\mathbf{x}(t)) = \nabla f(\mathbf{x}(t))^\top$. (2) By the chain rule, $D(f \circ g)(\mathbf{x}) = Df(g(\mathbf{x}))Dg(\mathbf{x}) = \nabla f(g(\mathbf{x}))^\top Dg(\mathbf{x})$; now take the transpose. □

Part (2) says that gradients are pulled back through a function by multiplying with the transposed Jacobian. This is the key step of backpropagation (Section 7).

Example 5.3. Let $f(x_1, x_2) = x_1^2 + 2x_2$, and let $\mathbf{x}(t) = (\sin t, \cos t)^\top$ move on the unit circle. Then $\nabla f(\mathbf{x}) = (2x_1, 2)^\top$ and $\mathbf{x}'(t) = (\cos t, -\sin t)^\top$, so

$$\frac{d}{dt} f(\mathbf{x}(t)) = 2\sin t \cos t - 2\sin t.$$

We can check this directly: $f(\mathbf{x}(t)) = \sin^2 t + 2\cos t$ has the derivative $2\sin t\cos t - 2\sin t$.

Corollary 5.4 (The gradient is orthogonal to level sets)**.** *Let $f : \mathbb{R}^n \to \mathbb{R}$ be differentiable, and let $\mathbf{x} : I \to \mathbb{R}^n$ be a differentiable curve in a level set of f, i.e., $f(\mathbf{x}(t)) = c$ for all $t \in I$. Then $\nabla f(\mathbf{x}(t))^\top \mathbf{x}'(t) = 0$ for all $t \in I$.*

Proof. By Corollary 5.2(1), $\nabla f(\mathbf{x}(t))^\top \mathbf{x}'(t)$ is the derivative of the constant function $t \mapsto c$, which is 0. □

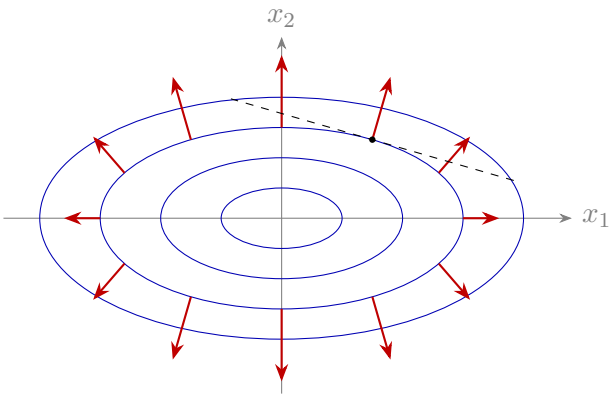


Figure 4: Level curves $x_1^2 + 4x_2^2 = c$ of $f(\mathbf{x}) = x_1^2 + 4x_2^2$ for $c = 1, 4, 9, 16$, and the gradients $\nabla f(\mathbf{x}) = (2x_1, 8x_2)^\top$ (scaled by 0.1) at points of the level curve $c = 9$. The gradients are orthogonal to the level curve (dashed: the tangent line at one point), and they are longest where the level curves are closest together, on the x_2-axis.

The vector $\mathbf{x}'(t)$ is tangent to the curve, so the gradient is orthogonal to every curve in the level set, i.e., orthogonal to the level set itself; see Figure 4. Together with Theorem 3.9, this gives a good picture: the steepest way up a hill is perpendicular to the contour lines of the map, and the gradient is longer where the contour lines are closer together.

Example 5.5 (The gradient of least squares)**.** Let $\Phi \in \mathbb{R}^{N\times d}$ be a design matrix and $\mathbf{y} \in \mathbb{R}^N$ a vector of targets, as in the section on least squares in Lecture 2, and let

$$L(\boldsymbol{\theta}) = \|\mathbf{y} - \Phi\boldsymbol{\theta}\|^2, \qquad \boldsymbol{\theta} \in \mathbb{R}^d.$$

We write $L = f \circ \mathbf{e}$ with the residual $\mathbf{e}(\boldsymbol{\theta}) = \mathbf{y} - \Phi\boldsymbol{\theta}$ and $f(\mathbf{e}) = \|\mathbf{e}\|^2$. By Example 4.4 and Proposition 3.3, $D\mathbf{e}(\boldsymbol{\theta}) = -\Phi$ and $Df(\mathbf{e}) = 2\mathbf{e}^\top$. By the chain rule,

$$DL(\boldsymbol{\theta}) = \underbrace{2(\mathbf{y} - \Phi\boldsymbol{\theta})^\top}_{1\times N}\,\underbrace{(-\Phi)}_{N\times d}, \qquad \text{so} \qquad \nabla L(\boldsymbol{\theta}) = -2\Phi^\top(\mathbf{y} - \Phi\boldsymbol{\theta}) = 2\Phi^\top(\Phi\boldsymbol{\theta} - \mathbf{y}).$$

Setting the gradient to zero gives $\Phi^\top\Phi\boldsymbol{\theta} = \Phi^\top\mathbf{y}$, the normal equations, which we found in Lecture 2 with orthogonal projections. Note how writing the sizes of the matrices under the factors checks that the product makes sense. This simple test catches many mistakes in gradient computations.

Example 5.6 (Logistic regression)**.** In binary classification, each data point $\mathbf{x} \in \mathbb{R}^n$ has a label $y \in \{0, 1\}$. Logistic regression predicts the probability $p = \sigma(\mathbf{w}^\top\mathbf{x})$ that $y = 1$, where σ is the sigmoid function, and it is trained by minimizing the cross-entropy loss, which for one data point is

$$\ell(\mathbf{w}) = -y \log p - (1 - y)\log(1 - p), \qquad p = \sigma(\mathbf{w}^\top\mathbf{x}).$$

It is small if the predicted probability of the correct label is close to 1. We compute the gradient with the chain rule, following the computation $\mathbf{w} \mapsto z = \mathbf{w}^\top\mathbf{x} \mapsto p = \sigma(z) \mapsto \ell$. By Example 2.4, $\frac{dp}{dz} = p(1 - p)$, so

$$\frac{d\ell}{dz} = \left(-\frac{y}{p} + \frac{1 - y}{1 - p}\right) p(1 - p) = -y(1 - p) + (1 - y)p = p - y,$$

and since $Dz(\mathbf{w}) = \mathbf{x}^\top$,

$$\nabla \ell(\mathbf{w}) = \big(\sigma(\mathbf{w}^\top \mathbf{x}) - y\big)\,\mathbf{x}:$$

the prediction error times the input. For example, for $\mathbf{x} = (1,2)^\top$, $y = 1$ and $\mathbf{w} = (0.5, -0.25)^\top$, we have $z = 0$, $p = \frac{1}{2}$, $\ell = \log 2$, and $\nabla \ell(\mathbf{w}) = -\frac{1}{2}(1,2)^\top = (-0.5, -1)^\top$. A step in the direction $-\nabla \ell(\mathbf{w})$ increases $\mathbf{w}^\top \mathbf{x}$, and hence the predicted probability of the correct label $y = 1$. For a data set $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)$ with the average loss $L(\mathbf{w}) = \frac{1}{N}\sum_i \ell_i(\mathbf{w})$, we get

$$\nabla L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} (\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i)\mathbf{x}_i = \frac{1}{N} X^\top (\sigma(X\mathbf{w}) - \mathbf{y}),$$

where $X \in \mathbb{R}^{N \times n}$ has the rows $\mathbf{x}_i^\top$, and σ is applied entrywise.

6 Gradients with Respect to Matrices

The parameters of a layer of a neural network include a weight matrix W, so we need derivatives of functions $f : \mathbb{R}^{m \times n} \to \mathbb{R}$. By Lecture 1, $\mathbb{R}^{m \times n}$ is a vector space isomorphic to $\mathbb{R}^{mn}$, so f is a function of the mn entries of X, and we can take the partial derivatives $\frac{\partial f}{\partial X_{ij}}$.

Definition 6.1 (Gradient with respect to a matrix)**.** Let $f : \mathbb{R}^{m \times n} \to \mathbb{R}$ be differentiable. The gradient of f at X is the $m \times n$ matrix $\nabla f(X)$ with the entries

$$\big(\nabla f(X)\big)_{ij} = \frac{\partial f}{\partial X_{ij}}(X).$$

The gradient has the same shape as X. Recall from Lecture 2 the Frobenius inner product $\langle A, B\rangle = \operatorname{tr}(A^\top B) = \sum_{i,j} A_{ij} B_{ij}$ on $\mathbb{R}^{m \times n}$, which is the dot product of the lists of entries, and the Frobenius norm $\|A\|_F = \sqrt{\langle A, A\rangle}$. Since f is a differentiable function of mn variables, Proposition 3.5 gives

$$f(X + H) = f(X) + \langle \nabla f(X), H\rangle + o(\|H\|_F),$$

and conversely, if $f(X + H) = f(X) + \langle G, H\rangle + o(\|H\|_F)$ for some matrix G, then $G = \nabla f(X)$. This gives a convenient method to compute gradients: expand $f(X + H)$, collect the terms that are linear in H, and write them in the form $\langle G, H\rangle = \operatorname{tr}(G^\top H)$. The following rules for the trace (Lecture 1) are useful: $\operatorname{tr}(A) = \operatorname{tr}(A^\top)$ and $\operatorname{tr}(AB) = \operatorname{tr}(BA)$, and hence

$$\operatorname{tr}(MH) = \operatorname{tr}(H^\top M^\top) = \langle M^\top, H\rangle \qquad \text{for all } M \in \mathbb{R}^{n \times m},\ H \in \mathbb{R}^{m \times n}.$$

Proposition 6.2. *Let $X \in \mathbb{R}^{m \times n}$. Then:*

1. $\nabla_X \operatorname{tr}(AX) = A^\top$ *for* $A \in \mathbb{R}^{n \times m}$, *and* $\nabla_X(\mathbf{a}^\top X \mathbf{b}) = \mathbf{a}\mathbf{b}^\top$ *for* $\mathbf{a} \in \mathbb{R}^m$, $\mathbf{b} \in \mathbb{R}^n$;
2. $\nabla_X \|AX - B\|_F^2 = 2A^\top(AX - B)$ *for* $A \in \mathbb{R}^{p \times m}$, $B \in \mathbb{R}^{p \times n}$; *in particular,* $\nabla_X \|X\|_F^2 = 2X$;
3. $\nabla_X \operatorname{tr}(X^\top A X) = (A + A^\top)X$ *for* $A \in \mathbb{R}^{m \times m}$.

Here the subscript in ∇_X indicates the variable.

Proof. (1) $\operatorname{tr}(A(X + H)) = \operatorname{tr}(AX) + \operatorname{tr}(AH)$, and $\operatorname{tr}(AH) = \langle A^\top, H\rangle$. For the second formula, $\mathbf{a}^\top X \mathbf{b}$ is a 1×1 matrix, so it equals its trace, and $\operatorname{tr}(\mathbf{a}^\top X \mathbf{b}) = \operatorname{tr}(\mathbf{b}\mathbf{a}^\top X)$; now use the first formula with $A = \mathbf{b}\mathbf{a}^\top$.

(2) Let $R = AX - B$. Then

$$\|A(X+H) - B\|_F^2 = \|R + AH\|_F^2 = \|R\|_F^2 + 2\langle R, AH\rangle + \|AH\|_F^2.$$

The last term is quadratic in H, so it is $o(\|H\|_F)$, and $\langle R, AH\rangle = \operatorname{tr}(R^\top AH) = \langle A^\top R, H\rangle$. For $A = I$ and $B = 0$, we get $\nabla\|X\|_F^2 = 2X$.

(3) $\operatorname{tr}((X+H)^\top A(X+H)) = \operatorname{tr}(X^\top AX) + \operatorname{tr}(H^\top AX) + \operatorname{tr}(X^\top AH) + \operatorname{tr}(H^\top AH)$. The last term is quadratic in H, $\operatorname{tr}(H^\top AX) = \langle H, AX\rangle = \langle AX, H\rangle$, and $\operatorname{tr}(X^\top AH) = \langle A^\top X, H\rangle$. □

For $n = 1$, part (3) is Proposition 3.3(2). The gradient of the determinant is a little harder to compute.

Lemma 6.3. *For every $E \in \mathbb{R}^{n\times n}$, $\det(I + tE) = 1 + t\operatorname{tr}(E) + t^2 q(t)$ for some polynomial q. In particular, $\frac{d}{dt}\det(I+tE)\big|_{t=0} = \operatorname{tr}(E)$.*

Proof. Let $\mu_1, \dots, \mu_n$ be the complex eigenvalues of E, so that $\det(E - \lambda I) = \prod_i(\mu_i - \lambda)$, and $\mu_1 + \cdots + \mu_n = \operatorname{tr}(E)$ by Lecture 1. For $t \neq 0$, we take $\lambda = -\frac{1}{t}$ and get

$$\det(I + tE) = t^n \det\Big(E + \frac{1}{t}I\Big) = t^n \prod_{i=1}^n \Big(\mu_i + \frac{1}{t}\Big) = \prod_{i=1}^n (1 + t\mu_i),$$

and this also holds for $t = 0$. Multiplying out the product gives

$$\det(I + tE) = 1 + t(\mu_1 + \cdots + \mu_n) + t^2 q(t) = 1 + t\operatorname{tr}(E) + t^2 q(t),$$

where q collects the terms with at least two factors $t\mu_i$. (Its coefficients are real, since $t^2 q(t)$ is the difference of the real polynomials $\det(I + tE)$ and $1 + t\operatorname{tr}(E)$.) □

Proposition 6.4 (Jacobi's formula)**.** *For every invertible matrix $X \in \mathbb{R}^{n\times n}$,*

$$\nabla_X \det(X) = \det(X)\, X^{-\top} \qquad \textit{and} \qquad \nabla_X \log|\det(X)| = X^{-\top},$$

where $X^{-\top} = (X^{-1})^\top$.

Proof. The determinant is a polynomial in the entries of X, so it is differentiable by Theorem 3.6. For every $H \in \mathbb{R}^{n\times n}$, by Lemma 6.3 with $E = X^{-1}H$,

$$\det(X + tH) = \det(X)\det(I + tX^{-1}H) = \det(X)\big(1 + t\operatorname{tr}(X^{-1}H) + t^2 q(t)\big).$$

So the directional derivative of det at X in the direction H is $\det(X)\operatorname{tr}(X^{-1}H) = \langle \det(X)X^{-\top}, H\rangle$. By Proposition 3.5, it is also equal to $\langle \nabla\det(X), H\rangle$. Taking for H the matrices with a single entry 1 and all other entries 0, we see that the two gradients have the same entries. For the second formula, $\frac{d}{du}\log|u| = \frac{1}{u}$ for $u \neq 0$, so by the chain rule $\nabla\log|\det(X)| = \frac{1}{\det(X)}\nabla\det(X) = X^{-\top}$. □

For $n = 1$, this says that $\frac{d}{dx}\log|x| = \frac{1}{x}$. There is also a direct way to see the first formula: by the cofactor expansion along the i-th row, $\frac{\partial\det(X)}{\partial X_{ij}}$ is the cofactor C_{ij}, and by Lecture 1, $X^{-1} = \frac{1}{\det(X)}C^\top$, i.e., $\det(X)X^{-\top} = C$. The gradient of $\log|\det|$ appears in the training of Gaussian models and normalizing flows (Remark 4.8).

Example 6.5 (A linear layer)**.** Let $\mathbf{y} = W\mathbf{x} + \mathbf{b}$ with $W \in \mathbb{R}^{m\times n}$, $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{b} \in \mathbb{R}^m$, and let L be a loss that depends on $\mathbf{y}$ (and maybe on other things). Let $\boldsymbol{\delta} = \nabla_{\mathbf{y}} L \in \mathbb{R}^m$ be the gradient of the loss with respect to the output of the layer. Then

$$\nabla_W L = \boldsymbol{\delta}\,\mathbf{x}^\top, \qquad \nabla_{\mathbf{b}} L = \boldsymbol{\delta}, \qquad \nabla_{\mathbf{x}} L = W^\top \boldsymbol{\delta}.$$

Indeed, $y_i = \sum_j W_{ij}x_j + b_i$, and the entry W_{ij} only influences L through y_i. So by the chain rule, $\frac{\partial L}{\partial W_{ij}} = \frac{\partial L}{\partial y_i} x_j = \delta_i x_j$, which is the (i,j) entry of the outer product $\boldsymbol{\delta}\mathbf{x}^\top$. Similarly, $\frac{\partial L}{\partial b_i} = \delta_i$, and the last formula is Corollary 5.2(2) with $D\mathbf{y}(\mathbf{x}) = W$. The sizes fit: $\boldsymbol{\delta}\mathbf{x}^\top$ is $m \times n$, like W.

In practice, a layer processes a batch of N inputs at once. If the inputs are the rows of $X \in \mathbb{R}^{N\times n}$, the outputs are the rows of $Y = XW^\top + \mathbf{1}\mathbf{b}^\top \in \mathbb{R}^{N\times m}$, and the loss is the sum of the losses of the data points, then the gradient is the sum of the gradients $\boldsymbol{\delta}_k \mathbf{x}_k^\top$ of the data points:

$$\nabla_W L = \sum_{k=1}^{N} \boldsymbol{\delta}_k \mathbf{x}_k^\top = \Delta^\top X, \qquad \nabla_{\mathbf{b}} L = \Delta^\top \mathbf{1},$$

where $\Delta \in \mathbb{R}^{N\times m}$ has the rows $\boldsymbol{\delta}_k^\top$.

Example 6.6 (Softmax with cross-entropy)**.** A classifier with K classes computes a vector of scores $\mathbf{z} \in \mathbb{R}^K$ and predicts the probabilities $\mathbf{p} = \mathrm{softmax}(\mathbf{z})$. If the correct class is c, the cross-entropy loss is

$$\ell(\mathbf{z}) = -\log p_c = -z_c + \log\left(e^{z_1} + \cdots + e^{z_K}\right).$$

Since $\frac{\partial}{\partial z_k} \log\left(\sum_j e^{z_j}\right) = \frac{e^{z_k}}{\sum_j e^{z_j}} = p_k$, the gradient is

$$\nabla \ell(\mathbf{z}) = \mathbf{p} - \mathbf{e}_c,$$

the predicted probabilities minus the "one-hot" vector of the correct class. This has the same form as for logistic regression. For example, for $\mathbf{z} = (2, 1, 0)^\top$ and $c = 1$, we get $\mathbf{p} \approx (0.665, 0.245, 0.090)^\top$, $\ell \approx 0.408$, and $\nabla\ell(\mathbf{z}) \approx (-0.335, 0.245, 0.090)^\top$. Gradient descent increases the score of the correct class and decreases the other scores. If the scores come from a linear layer $\mathbf{z} = W\mathbf{x} + \mathbf{b}$, then by Example 6.5, $\nabla_W \ell = (\mathbf{p} - \mathbf{e}_c)\mathbf{x}^\top$.

We also found that the gradient of the log-sum-exp function $\log(e^{z_1} + \cdots + e^{z_K})$ is $\mathrm{softmax}(\mathbf{z})$. Table 1 collects the gradients of this and the previous sections.

Remark 6.7 (Derivatives of matrix-valued functions)*.* The derivative of a function $F : \mathbb{R}^{m\times n} \to \mathbb{R}^{p\times q}$ at X is a linear map from $\mathbb{R}^{m\times n}$ to $\mathbb{R}^{p\times q}$. Its matrix with respect to the standard bases is a $pq \times mn$ matrix, which is sometimes arranged as a four-dimensional array of numbers (a tensor). In machine learning, we rarely need these large objects explicitly: we only need the gradient of a scalar loss, and the chain rule lets us pass gradients through each function without ever writing down its full Jacobian, as in Example 6.5. This is how backpropagation works, which is the topic of the next section.

7 Backpropagation and Automatic Differentiation

To train a neural network, we need the gradient of the loss with respect to millions of parameters, at every step of training. There are three ways to let a computer compute derivatives.

function	gradient	assumptions
$\mathbf{a}^\top \mathbf{x}$	$\mathbf{a}$	
$\mathbf{x}^\top A \mathbf{x}$	$(A + A^\top)\mathbf{x}$	$(= 2A\mathbf{x}$ if A is symmetric)
$\|\mathbf{x}\|^2$	$2\mathbf{x}$	
$\|\mathbf{x}\|$	$\mathbf{x}/\|\mathbf{x}\|$	$\mathbf{x} \neq \mathbf{0}$
$\|A\mathbf{x} - \mathbf{b}\|^2$	$2A^\top(A\mathbf{x} - \mathbf{b})$	
$f(g(\mathbf{x}))$	$Dg(\mathbf{x})^\top \nabla f(g(\mathbf{x}))$	f scalar-valued
$\log(e^{z_1} + \cdots + e^{z_K})$	$\operatorname{softmax}(\mathbf{z})$	
$-\log \operatorname{softmax}(\mathbf{z})_c$	$\operatorname{softmax}(\mathbf{z}) - \mathbf{e}_c$	
$\operatorname{tr}(AX)$	$A^\top$	
$\mathbf{a}^\top X \mathbf{b}$	$\mathbf{a}\mathbf{b}^\top$	
$\|AX - B\|_F^2$	$2A^\top(AX - B)$	
$\operatorname{tr}(X^\top A X)$	$(A + A^\top)X$	
$\det(X)$	$\det(X)X^{-\top}$	X invertible
$\log \lvert\det(X)\rvert$	$X^{-\top}$	X invertible

Table 1: Some gradients with respect to a vector $\mathbf{x}$ or $\mathbf{z}$, or a matrix X.

- Symbolic differentiation, as in computer algebra systems, applies the rules of differentiation to formulas. The resulting formulas can become huge, and many programs, with loops and if-statements, are not given by a single formula.
- Numerical differentiation uses finite differences (Proposition 2.9). For a function of n variables, the gradient needs at least $n + 1$ evaluations of the function, which is far too slow for $n = 10^6$, and the result is only approximate.
- Automatic differentiation (AD) uses the fact that every program computes a function by a sequence of elementary operations, such as $+$, $\times$, exp and log, whose derivatives are known. It applies the chain rule to this sequence, with numbers instead of formulas. The result is exact up to rounding errors, and in reverse mode, the cost of the gradient is only a small multiple of the cost of evaluating the function.

Backpropagation is reverse-mode automatic differentiation, applied to neural networks.

7.1 Forward mode and reverse mode

Let $f = f_L \circ \cdots \circ f_2 \circ f_1$ be a composition of functions $f_l : \mathbb{R}^{n_{l-1}} \to \mathbb{R}^{n_l}$, with $n_0 = n$ and $n_L = m$, such as the layers of a neural network. We evaluate f step by step: $\mathbf{x}_0 = \mathbf{x}$ and $\mathbf{x}_l = f_l(\mathbf{x}_{l-1})$ for $l = 1, \ldots, L$. By the chain rule,

$$Df(\mathbf{x}) = J_L J_{L-1} \cdots J_1, \qquad \text{where } J_l = Df_l(\mathbf{x}_{l-1}) \in \mathbb{R}^{n_l \times n_{l-1}}.$$

Matrix multiplication is associative, so we can compute this product in any order, but the cost depends on the order: multiplying a $p \times q$ matrix by a $q \times r$ matrix costs pqr multiplications.

- Forward mode computes $J_L(\cdots(J_2(J_1\mathbf{v})))$ for a vector $\mathbf{v} \in \mathbb{R}^n$, i.e., the Jacobian-vector product $Df(\mathbf{x})\mathbf{v}$, from right to left. Each multiplication can be done together with the evaluation of f_l, in the forward direction. For $\mathbf{v} = \mathbf{e}_j$, we get the j-th column of $Df(\mathbf{x})$, so the full Jacobian needs n passes.
- Reverse mode computes $((\mathbf{u}^\top J_L)J_{L-1})\cdots J_1$ for a row vector $\mathbf{u}^\top \in \mathbb{R}^{1\times m}$, i.e., the vector-Jacobian product $\mathbf{u}^\top Df(\mathbf{x})$, from left to right. This goes backwards, from the output to the input, so we first evaluate f and store the intermediate values $\mathbf{x}_0, \ldots, \mathbf{x}_{L-1}$, which we need to compute the Jacobians. For $\mathbf{u} = \mathbf{e}_i$, we get the i-th row of $Df(\mathbf{x})$, so the full Jacobian needs m passes.

For a loss function, $m = 1$, and a single backward pass with $\mathbf{u} = 1$ gives the whole gradient. For example, if $n_0 = \cdots = n_{L-1} = n$ and $n_L = 1$, then computing the gradient in forward mode needs n passes, each with about Ln^2 multiplications, so about Ln^3 in total. Reverse mode needs one pass with L vector-matrix products, about Ln^2 multiplications, which is comparable to the cost of evaluating f if the f_l are linear layers. For $n = 1000$ and $L = 10$, this is 10^{10} versus 10^7 multiplications. In general, the cost of reverse mode for a scalar function is a small multiple (typically between 2 and 5) of the cost of evaluating the function, no matter how many variables it has. The price is memory, since all intermediate values have to be stored. In the language of Lecture 1, reverse mode applies the transposes, or dual maps, $J_1^\top \cdots J_L^\top \mathbf{u}$, which is the rule $(S \circ T)^* = T^* \circ S^*$.

7.2 Computation graphs

Most programs are not a simple chain of functions. In general, every intermediate variable v_j is computed from some earlier variables by an elementary operation, which we draw as a computation graph, with an arrow $v_i \to v_j$ if v_j is computed from v_i. The two modes of automatic differentiation work as follows.

- Forward mode: choose a direction $\dot{\mathbf{x}} = \mathbf{v}$, and compute the derivatives $\dot{v}_j$ of all intermediate variables in this direction, in the order of evaluation:

$$\dot{v}_j = \sum_{i:v_i \to v_j} \frac{\partial v_j}{\partial v_i} \dot{v}_i.$$

- Reverse mode: first evaluate all v_j, and then compute the adjoints $\bar{v}_i = \frac{\partial f}{\partial v_i}$, which describe how the output depends on v_i, in the reverse order, starting with $\bar{f} = 1$:

$$\bar{v}_i = \sum_{j:v_i \to v_j} \bar{v}_j \frac{\partial v_j}{\partial v_i}.$$

  If v_i is used by several later variables, their contributions are added; this is the sum in the chain rule (Theorem 5.1).

Example 7.1. Let $f(x_1, x_2) = \log x_1 + x_1 x_2 - \sin x_2$. Figure 5 shows how a computer evaluates f, and Table 2 shows both modes of automatic differentiation at $(x_1, x_2) = (2, 5)$. Forward mode with the direction $\dot{\mathbf{x}} = \mathbf{e}_1$ gives $\frac{\partial f}{\partial x_1} = 5.5$; for $\frac{\partial f}{\partial x_2}$, we need a second pass with $\dot{\mathbf{x}} = \mathbf{e}_2$. Reverse mode gives both partial derivatives in one pass. Note that $\bar{x}_1$ collects contributions from v_1 and v_2, since x_1 is used to compute both. We check the result: $\frac{\partial f}{\partial x_1} = \frac{1}{x_1} + x_2 = 5.5$ and $\frac{\partial f}{\partial x_2} = x_1 - \cos x_2 \approx 2 - 0.284 = 1.716$.

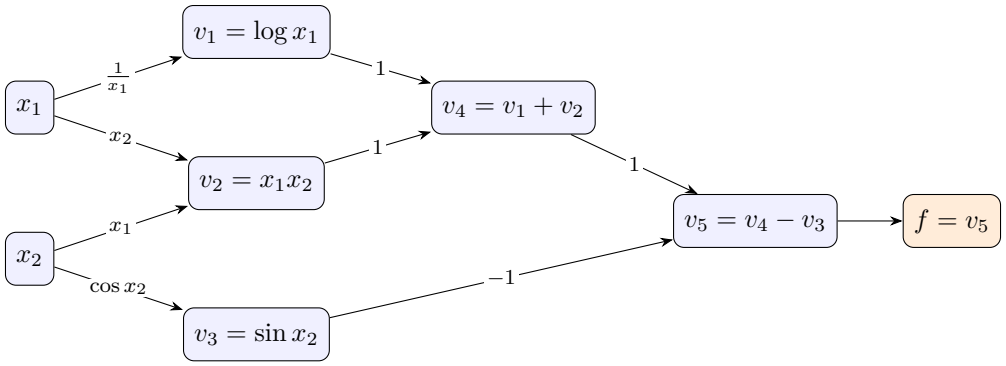


Figure 5: The computation graph of $f(x_1, x_2) = \log x_1 + x_1 x_2 - \sin x_2$. Each arrow $v_i \to v_j$ is labeled with the local partial derivative $\frac{\partial v_j}{\partial v_i}$.

evaluation		forward mode, $\dot{\mathbf{x}} = \mathbf{e}_1$	reverse mode
$x_1 = 2$		$\dot{x}_1 = 1$	$\bar{v}_5 = 1$
$x_2 = 5$		$\dot{x}_2 = 0$	$\bar{v}_4 = \bar{v}_5 = 1$
$v_1 = \log x_1$	$= 0.693$	$\dot{v}_1 = \dot{x}_1/x_1 = 0.5$	$\bar{v}_3 = -\bar{v}_5 = -1$
$v_2 = x_1 x_2$	$= 10$	$\dot{v}_2 = \dot{x}_1 x_2 + x_1 \dot{x}_2 = 5$	$\bar{v}_2 = \bar{v}_4 = 1$
$v_3 = \sin x_2$	$= -0.959$	$\dot{v}_3 = \cos(x_2)\, \dot{x}_2 = 0$	$\bar{v}_1 = \bar{v}_4 = 1$
$v_4 = v_1 + v_2$	$= 10.693$	$\dot{v}_4 = \dot{v}_1 + \dot{v}_2 = 5.5$	$\bar{x}_2 = \bar{v}_2 x_1 + \bar{v}_3 \cos x_2 = 1.716$
$v_5 = v_4 - v_3$	$= 11.652$	$\dot{v}_5 = \dot{v}_4 - \dot{v}_3 = 5.5$	$\bar{x}_1 = \bar{v}_1/x_1 + \bar{v}_2 x_2 = 5.5$

Table 2: Automatic differentiation of $f(x_1, x_2) = \log x_1 + x_1 x_2 - \sin x_2$ at $(2, 5)$. Left: evaluation and forward mode, from top to bottom. Right: reverse mode, from top to bottom, i.e., backwards through the computation graph.

7.3 Backpropagation in a neural network

Consider a neural network with one hidden layer, which maps an input $\mathbf{x} \in \mathbb{R}^n$ to an output $\hat{\mathbf{y}} \in \mathbb{R}^m$, and its squared loss for a target $\mathbf{y} \in \mathbb{R}^m$:

$$\mathbf{z} = W_1 \mathbf{x} + \mathbf{b}_1, \qquad \mathbf{a} = \sigma(\mathbf{z}), \qquad \hat{\mathbf{y}} = W_2 \mathbf{a} + \mathbf{b}_2, \qquad L = \tfrac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|^2,$$

where $W_1 \in \mathbb{R}^{h \times n}$, $\mathbf{b}_1 \in \mathbb{R}^h$, $W_2 \in \mathbb{R}^{m \times h}$, $\mathbf{b}_2 \in \mathbb{R}^m$, and σ is applied entrywise. The forward pass computes and stores $\mathbf{z}$, $\mathbf{a}$, $\hat{\mathbf{y}}$ and L. The backward pass computes, using Examples 6.5 and 4.5:

$$\begin{aligned}
\boldsymbol{\delta}_2 &= \nabla_{\hat{\mathbf{y}}} L = \hat{\mathbf{y}} - \mathbf{y}, \\
\nabla_{W_2} L &= \boldsymbol{\delta}_2 \mathbf{a}^\top, \qquad \nabla_{\mathbf{b}_2} L = \boldsymbol{\delta}_2, \\
\nabla_{\mathbf{a}} L &= W_2^\top \boldsymbol{\delta}_2, \\
\boldsymbol{\delta}_1 &= \nabla_{\mathbf{z}} L = D\sigma(\mathbf{z})^\top \nabla_{\mathbf{a}} L = \sigma'(\mathbf{z}) \odot (W_2^\top \boldsymbol{\delta}_2), \\
\nabla_{W_1} L &= \boldsymbol{\delta}_1 \mathbf{x}^\top, \qquad \nabla_{\mathbf{b}_1} L = \boldsymbol{\delta}_1.
\end{aligned}$$

Each line only uses values from the forward pass and from the previous lines. For a deeper network with layers $\mathbf{z}_l = W_l \mathbf{a}_{l-1} + \mathbf{b}_l$, $\mathbf{a}_l = \sigma(\mathbf{z}_l)$, the same pattern repeats: the "error signals" $\boldsymbol{\delta}_l = \nabla_{\mathbf{z}_l} L$

satisfy

$$\boldsymbol{\delta}_l = \sigma'(\mathbf{z}_l) \odot (W_{l+1}^\top \boldsymbol{\delta}_{l+1}), \qquad \nabla_{W_l} L = \boldsymbol{\delta}_l \mathbf{a}_{l-1}^\top, \qquad \nabla_{\mathbf{b}_l} L = \boldsymbol{\delta}_l.$$

The error signals are propagated backwards through the network, which explains the name backpropagation. The backward pass of a layer costs about twice as much as its forward pass: it computes $W_{l+1}^\top \boldsymbol{\delta}_{l+1}$ and the outer product $\boldsymbol{\delta}_l \mathbf{a}_{l-1}^\top$, and each of them costs about as much as the product $W_l \mathbf{a}_{l-1}$ in the forward pass. Machine learning libraries such as PyTorch and JAX implement reverse-mode automatic differentiation for general programs: they record the computation graph during the forward pass, and each elementary operation knows how to compute its vector-Jacobian product.

Remark 7.2 (Vanishing and exploding gradients). By the recursion above, $\boldsymbol{\delta}_1$ is obtained from $\boldsymbol{\delta}_L$ by multiplying with the matrices $D\sigma(\mathbf{z}_l)^\top W_{l+1}^\top = \operatorname{diag}(\sigma'(\mathbf{z}_l)) W_{l+1}^\top$, once for every layer. For the sigmoid function, $\sigma' \le \frac{1}{4}$ (Example 2.4), so unless the weights are large, the gradients of the first layers become tiny in deep networks (vanishing gradients), and these layers hardly learn. If the weight matrices have large norms, the gradients can instead grow exponentially (exploding gradients). Remedies include the ReLU activation function (whose derivative is 1 for active units), careful initialization of the weights, and residual connections $\mathbf{a}_l = \mathbf{a}_{l-1} + f_l(\mathbf{a}_{l-1})$, whose Jacobian $I + Df_l$ is close to the identity matrix.

Remark 7.3 (Gradient checking). Hand-written backward passes are easy to get wrong. To check them, compare the computed gradient $\mathbf{g}$ with central differences $\tilde{g}_i = \frac{f(\mathbf{x}+h\mathbf{e}_i) - f(\mathbf{x}-h\mathbf{e}_i)}{2h}$ for a few random inputs, and compute the relative error $\frac{\|\mathbf{g}-\tilde{\mathbf{g}}\|}{\|\mathbf{g}\|+\|\tilde{\mathbf{g}}\|}$. With $h \approx 10^{-5}$, a relative error around 10^{-8} or smaller indicates a correct gradient, while a mistake typically gives a relative error of order 1.

8 Higher Derivatives and Taylor's Theorem

The gradient describes how a function changes to first order. To see whether a critical point is a minimum, and how fast optimization algorithms converge, we need the second derivatives, which describe the curvature of the function.

Definition 8.1 (Hessian matrix). Let $f : D \to \mathbb{R}$ with $D \subseteq \mathbb{R}^n$ open. If the partial derivative $\frac{\partial f}{\partial x_i}$ has a partial derivative with respect to x_j, we write

$$\frac{\partial^2 f}{\partial x_j \partial x_i} = \frac{\partial}{\partial x_j}\left(\frac{\partial f}{\partial x_i}\right), \qquad \frac{\partial^2 f}{\partial x_i^2} = \frac{\partial}{\partial x_i}\left(\frac{\partial f}{\partial x_i}\right).$$

The function f is twice continuously differentiable if all these second partial derivatives exist and are continuous on D. The Hessian matrix of f at $\mathbf{x}$ is the $n \times n$ matrix $\nabla^2 f(\mathbf{x})$ with the entries

$$\left(\nabla^2 f(\mathbf{x})\right)_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}(\mathbf{x}).$$

Theorem 8.2 (Schwarz). *If f is twice continuously differentiable on D, then $\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i}$ for all i, j. In particular, the Hessian matrix is symmetric.*

We do not prove this theorem. The Hessian is the Jacobian of the gradient, $\nabla^2 f(\mathbf{x}) = D(\nabla f)(\mathbf{x})$: the i-th row of $D(\nabla f)(\mathbf{x})$ is the derivative of $\frac{\partial f}{\partial x_i}$, with the entries $\frac{\partial^2 f}{\partial x_j \partial x_i}$, which are equal to $\frac{\partial^2 f}{\partial x_i \partial x_j}$ by Schwarz's theorem.

Example 8.3. For $f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3$ (Example 3.2), we have $\frac{\partial f}{\partial x_1} = 2x_1x_2 + x_2^3$ and $\frac{\partial f}{\partial x_2} = x_1^2 + 3x_1x_2^2$, so

$$\nabla^2 f(\mathbf{x}) = \begin{pmatrix} 2x_2 & 2x_1 + 3x_2^2 \\ 2x_1 + 3x_2^2 & 6x_1x_2 \end{pmatrix}, \qquad \nabla^2 f(1,2) = \begin{pmatrix} 4 & 14 \\ 14 & 12 \end{pmatrix}.$$

As predicted by Schwarz's theorem, the two mixed partial derivatives are equal.

Example 8.4 (Hessians in machine learning)**.**

1. For a quadratic function $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top A\mathbf{x} - \mathbf{b}^\top\mathbf{x} + c$ with a symmetric matrix A, Proposition 3.3 gives $\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$, and by Example 4.4, $\nabla^2 f(\mathbf{x}) = A$ for all $\mathbf{x}$. For example, the least squares loss $L(\boldsymbol{\theta}) = \|\mathbf{y} - \Phi\boldsymbol{\theta}\|^2$ has the gradient $2\Phi^\top\Phi\boldsymbol{\theta} - 2\Phi^\top\mathbf{y}$ and the Hessian $2\Phi^\top\Phi$.

2. For logistic regression (Example 5.6), $\nabla\ell(\mathbf{w}) = (\sigma(\mathbf{w}^\top\mathbf{x}) - y)\mathbf{x}$. Its Jacobian is $\mathbf{x}\cdot\sigma'(\mathbf{w}^\top\mathbf{x})\mathbf{x}^\top$, so $\nabla^2\ell(\mathbf{w}) = p(1-p)\mathbf{x}\mathbf{x}^\top$ with $p = \sigma(\mathbf{w}^\top\mathbf{x})$. For a data set, $\nabla^2 L(\mathbf{w}) = \frac{1}{N}\sum_i p_i(1-p_i)\mathbf{x}_i\mathbf{x}_i^\top = \frac{1}{N}X^\top SX$, where $S = \operatorname{diag}(p_1(1-p_1), \ldots, p_N(1-p_N))$.

3. The log-sum-exp function $\operatorname{lse}(\mathbf{z}) = \log(e^{z_1} + \cdots + e^{z_K})$ has the gradient $\operatorname{softmax}(\mathbf{z})$ (Example 6.6), so its Hessian is the Jacobian of the softmax function, $\nabla^2 \operatorname{lse}(\mathbf{z}) = \operatorname{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$ with $\mathbf{p} = \operatorname{softmax}(\mathbf{z})$ (Example 4.6).

8.1 Taylor's theorem in several variables

Theorem 8.5 (Second-order Taylor expansion)**.** *Let f be twice continuously differentiable on an open set D that contains the line segment from $\mathbf{x}$ to $\mathbf{x} + \mathbf{h}$. Then there is a $\tau \in (0,1)$ such that*

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top\mathbf{h} + \frac{1}{2}\mathbf{h}^\top\nabla^2 f(\mathbf{x} + \tau\mathbf{h})\,\mathbf{h}.$$

Consequently,

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top\mathbf{h} + \frac{1}{2}\mathbf{h}^\top\nabla^2 f(\mathbf{x})\,\mathbf{h} + o(\|\mathbf{h}\|^2).$$

Proof. We reduce to one variable. Let $g(t) = f(\mathbf{x} + t\mathbf{h})$ for t in an open interval containing $[0,1]$. By the chain rule (Corollary 5.2(1)),

$$g'(t) = \nabla f(\mathbf{x} + t\mathbf{h})^\top\mathbf{h} = \sum_{i=1}^n \frac{\partial f}{\partial x_i}(\mathbf{x} + t\mathbf{h})\,h_i,$$

and applying the chain rule again to each $\frac{\partial f}{\partial x_i}$,

$$g''(t) = \sum_{i=1}^n\sum_{j=1}^n \frac{\partial^2 f}{\partial x_j\partial x_i}(\mathbf{x} + t\mathbf{h})\,h_jh_i = \mathbf{h}^\top\nabla^2 f(\mathbf{x} + t\mathbf{h})\,\mathbf{h}.$$

By Taylor's theorem in one variable (Theorem 2.6 with $k = 1$), $g(1) = g(0) + g'(0) + \frac{1}{2}g''(\tau)$ for some $\tau \in (0,1)$, which is the first formula. For the second formula, the difference between the two quadratic terms is

$$\tfrac{1}{2}|\mathbf{h}^\top(\nabla^2 f(\mathbf{x} + \tau\mathbf{h}) - \nabla^2 f(\mathbf{x}))\mathbf{h}| \le \tfrac{1}{2}\|\nabla^2 f(\mathbf{x} + \tau\mathbf{h}) - \nabla^2 f(\mathbf{x})\|_2\|\mathbf{h}\|^2$$

by the Cauchy–Schwarz inequality, and the spectral norm tends to 0 as $\mathbf{h} \to \mathbf{0}$, since the second partial derivatives are continuous (and the spectral norm is at most the Frobenius norm). □

So near $\mathbf{x}$, f is approximated by the quadratic function

$$f(\mathbf{x}+\mathbf{h}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{h} + \tfrac{1}{2}\mathbf{h}^\top \nabla^2 f(\mathbf{x})\mathbf{h}.$$

The gradient gives the slope, and the Hessian gives the curvature: for a unit vector $\mathbf{v}$, the number $\mathbf{v}^\top \nabla^2 f(\mathbf{x})\mathbf{v}$ is the second derivative of $t \mapsto f(\mathbf{x}+t\mathbf{v})$ at $t=0$, i.e., the curvature of f in the direction $\mathbf{v}$. The eigenvalues and eigenvectors of the Hessian describe the curvature in all directions at once; we work this out in Section 9.1, where it is the key to classifying critical points.

Example 8.6. Let $f(x,y) = e^x \cos y$. Then

$$\nabla f(x,y) = \begin{pmatrix} e^x \cos y \\ -e^x \sin y \end{pmatrix}, \qquad \nabla^2 f(x,y) = \begin{pmatrix} e^x \cos y & -e^x \sin y \\ -e^x \sin y & -e^x \cos y \end{pmatrix},$$

so at the origin, $f = 1$, $\nabla f = (1,0)^\top$ and $\nabla^2 f = \operatorname{diag}(1,-1)$. The second-order approximation is

$$e^x \cos y \approx 1 + x + \tfrac{1}{2}(x^2 - y^2).$$

At $(0.1, 0.2)$, it gives 1.085, while the linear approximation $1+x$ gives 1.1, and the exact value is $1.0831\ldots$.

There are also Taylor expansions of higher order, which use all partial derivatives of order k. They are rarely needed in machine learning, and we do not discuss them.

9 Unconstrained Optimization

We now use derivatives to minimize functions, which is how models are trained. Maximizing f is the same as minimizing $-f$, so we only discuss minimization.

Definition 9.1 (Minima and critical points)**.** Let $f : D \to \mathbb{R}$ with $D \subseteq \mathbb{R}^n$ open.

1. A point $\mathbf{x}^* \in D$ is a global minimum (or global minimizer) of f if $f(\mathbf{x}^*) \le f(\mathbf{x})$ for all $\mathbf{x} \in D$, and a local minimum if there is an $r > 0$ such that $f(\mathbf{x}^*) \le f(\mathbf{x})$ for all $\mathbf{x} \in D$ with $\|\mathbf{x} - \mathbf{x}^*\| < r$. It is a strict local minimum if moreover $f(\mathbf{x}^*) < f(\mathbf{x})$ for these $\mathbf{x} \neq \mathbf{x}^*$. Maxima are defined in the same way.

2. If f is differentiable, a point $\mathbf{x}^*$ with $\nabla f(\mathbf{x}^*) = \mathbf{0}$ is called a critical point (or stationary point) of f.

We write $\arg\min_{\mathbf{x}} f(\mathbf{x})$ for the set of global minima of f, or for the global minimum if it is unique.

Theorem 9.2 (First-order condition)**.** *If f is differentiable and $\mathbf{x}^*$ is a local minimum of f, then $\nabla f(\mathbf{x}^*) = \mathbf{0}$.*

Proof. Suppose that $\mathbf{g} = \nabla f(\mathbf{x}^*) \neq \mathbf{0}$. By Proposition 3.5, the directional derivative of f at $\mathbf{x}^*$ in the direction $-\mathbf{g}$ is $\nabla f(\mathbf{x}^*)^\top(-\mathbf{g}) = -\|\mathbf{g}\|^2 < 0$. So $\frac{f(\mathbf{x}^* - t\mathbf{g}) - f(\mathbf{x}^*)}{t} < 0$ for all small $t > 0$, i.e., $f(\mathbf{x}^* - t\mathbf{g}) < f(\mathbf{x}^*)$, and $\mathbf{x}^*$ is not a local minimum. □

The same holds for local maxima. The converse is false: a critical point can be a local minimum, a local maximum, or neither. For example, the origin is a critical point of $g(x,y) = x^2 - y^2$ (Figure 1), but g increases along the x-axis and decreases along the y-axis. A critical point that is neither a local minimum nor a local maximum is called a saddle point. To tell these cases apart, we look at the second-order term of the Taylor expansion, and in particular at the eigenvalues of the Hessian.

9.1 The second-order test

Let $\nabla f(\mathbf{x}^*) = \mathbf{0}$, and let $H = \nabla^2 f(\mathbf{x}^*)$. Since the gradient vanishes, Theorem 8.5 gives

$$f(\mathbf{x}^* + \mathbf{h}) = f(\mathbf{x}^*) + \tfrac{1}{2}\mathbf{h}^\top H \mathbf{h} + o(\|\mathbf{h}\|^2),$$

so near $\mathbf{x}^*$, the shape of f is described by the quadratic form $\mathbf{h}^\top H \mathbf{h}$. The Hessian is symmetric, so by the spectral theorem (Lecture 2) it has an orthonormal basis of eigenvectors $\mathbf{q}_1, \dots, \mathbf{q}_n$, with real eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n$, and $H = Q\Lambda Q^\top$. Let us measure the displacement $\mathbf{h}$ in this basis: $\mathbf{h} = y_1\mathbf{q}_1 + \cdots + y_n\mathbf{q}_n$, i.e., $\mathbf{y} = Q^\top \mathbf{h}$. Then $H\mathbf{h} = \sum_i \lambda_i y_i \mathbf{q}_i$, and since the $\mathbf{q}_i$ are orthonormal,

$$\mathbf{h}^\top H \mathbf{h} = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2 \qquad \text{and} \qquad \|\mathbf{h}\|^2 = y_1^2 + \cdots + y_n^2.$$

Therefore, near the critical point,

$$f(\mathbf{x}^* + y_1\mathbf{q}_1 + \cdots + y_n\mathbf{q}_n) \approx f(\mathbf{x}^*) + \tfrac{1}{2}(\lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2).$$

In the coordinates given by the eigenvectors, the mixed terms $y_i y_j$ disappear: near a critical point, f looks like a sum of n independent parabolas, one for each eigenvector. Along the line through $\mathbf{x}^*$ in the direction $\mathbf{q}_i$,

$$f(\mathbf{x}^* + t\mathbf{q}_i) \approx f(\mathbf{x}^*) + \tfrac{1}{2}\lambda_i t^2,$$

a parabola that opens upward if $\lambda_i > 0$ and downward if $\lambda_i < 0$. So the eigenvalue λ_i is the curvature of f in the direction $\mathbf{q}_i$, i.e., the second derivative of $t \mapsto f(\mathbf{x}^* + t\mathbf{q}_i)$ at $t = 0$. The eigenvectors are called the principal directions (or principal axes) of f at $\mathbf{x}^*$. In any other unit direction $\mathbf{v}$, with $\mathbf{y} = Q^\top \mathbf{v}$, the curvature is $\mathbf{v}^\top H \mathbf{v} = \sum_i \lambda_i y_i^2$ with $\sum_i y_i^2 = 1$, a weighted average of the eigenvalues; so it lies between λ_n and λ_1, as for the Rayleigh quotient in Lecture 2.

Hence the signs of the eigenvalues determine the shape of f near a critical point (Figure 6 and Table 3). If all eigenvalues are positive, f curves upward in every direction, like a bowl, and $\mathbf{x}^*$ is a minimum. If all are negative, f curves downward in every direction, like an upside-down bowl, and $\mathbf{x}^*$ is a maximum. If there are eigenvalues of both signs, f curves upward in some directions and downward in others, like a saddle or a mountain pass. And if some eigenvalues are 0, then f is flat to second order in these directions, like a trough, and the quadratic approximation cannot decide what happens: this depends on the terms of higher order.

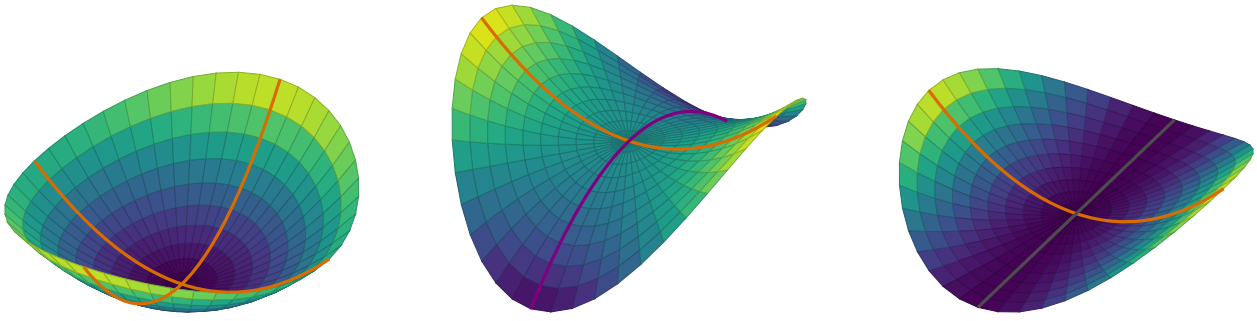


Figure 6: The quadratic approximation $f(\mathbf{x}^*) + \frac{1}{2}(\lambda_1 y_1^2 + \lambda_2 y_2^2)$ of a function near a critical point, in the coordinates y_1, y_2 along the eigenvectors of the Hessian. The curves are the parabolas $\frac{1}{2}\lambda_i t^2$ along the eigenvectors: orange for $\lambda_i > 0$, violet for $\lambda_i < 0$, and gray for $\lambda_i = 0$. Left: $\lambda_1, \lambda_2 > 0$, a bowl (local minimum). Middle: $\lambda_1 > 0 > \lambda_2$, a saddle (saddle point). Right: $\lambda_1 > 0$ and $\lambda_2 = 0$, a trough (the second-order test is inconclusive).

Theorem 9.3 (Second-order test)**.** *Let f be twice continuously differentiable on D, let $\nabla f(\mathbf{x}^*) = \mathbf{0}$, and let $\lambda_1 \geq \cdots \geq \lambda_n$ be the eigenvalues of the Hessian $\nabla^2 f(\mathbf{x}^*)$.*

1. *If all eigenvalues are positive ($\lambda_n > 0$), then $\mathbf{x}^*$ is a strict local minimum.*
2. *If all eigenvalues are negative ($\lambda_1 < 0$), then $\mathbf{x}^*$ is a strict local maximum.*
3. *If there are positive and negative eigenvalues ($\lambda_1 > 0 > \lambda_n$), then $\mathbf{x}^*$ is a saddle point: f increases along every eigenvector with a positive eigenvalue, and decreases along every eigenvector with a negative eigenvalue.*
4. *Conversely, if $\mathbf{x}^*$ is a local minimum, then all eigenvalues are ≥ 0, and if $\mathbf{x}^*$ is a local maximum, then all eigenvalues are ≤ 0.*

Proof. Let $H = \nabla^2 f(\mathbf{x}^*)$, and write $f(\mathbf{x}^* + \mathbf{h}) = f(\mathbf{x}^*) + \frac{1}{2}\mathbf{h}^\top H \mathbf{h} + r(\mathbf{h})$ with $r(\mathbf{h}) = o(\|\mathbf{h}\|^2)$, as above.

(1) In the coordinates $\mathbf{y} = Q^\top \mathbf{h}$, we have $\mathbf{h}^\top H \mathbf{h} = \sum_i \lambda_i y_i^2 \geq \lambda_n \sum_i y_i^2 = \lambda_n \|\mathbf{h}\|^2$: in every direction, f curves upward at least as much as along $\mathbf{q}_n$. For all sufficiently small $\mathbf{h} \neq \mathbf{0}$, we have $|r(\mathbf{h})| \leq \frac{1}{4}\lambda_n \|\mathbf{h}\|^2$, and hence $f(\mathbf{x}^* + \mathbf{h}) \geq f(\mathbf{x}^*) + \frac{1}{4}\lambda_n \|\mathbf{h}\|^2 > f(\mathbf{x}^*)$.

(2) Apply (1) to $-f$, whose Hessian has the eigenvalues $-\lambda_i$.

(3) Let $\mathbf{q}$ be a unit eigenvector with eigenvalue $\lambda \neq 0$. Then $f(\mathbf{x}^* + t\mathbf{q}) = f(\mathbf{x}^*) + \frac{1}{2}\lambda t^2 + r(t\mathbf{q})$, and $|r(t\mathbf{q})| < \frac{1}{2}|\lambda| t^2$ for all small $t \neq 0$. So for all small $t \neq 0$, $f(\mathbf{x}^* + t\mathbf{q}) > f(\mathbf{x}^*)$ if $\lambda > 0$, and $f(\mathbf{x}^* + t\mathbf{q}) < f(\mathbf{x}^*)$ if $\lambda < 0$. If both signs occur, there are points with larger and points with smaller values arbitrarily close to $\mathbf{x}^*$, so $\mathbf{x}^*$ is neither a local minimum nor a local maximum.

(4) If some eigenvalue were negative, then by the argument of (3), there would be points with smaller values arbitrarily close to $\mathbf{x}^*$, so $\mathbf{x}^*$ would not be a local minimum. The case of a local maximum is the same. □

In the language of Lecture 2, the Hessian in (1) is positive definite, in (2) negative definite, and in (3) indefinite, and (4) says that the Hessian at a local minimum is positive semidefinite. If some eigenvalues are 0 and all others have the same sign, the test is inconclusive: x^4, $-x^4$ and x^3 all have the critical point 0 with second derivative 0, but the first has a minimum there, the second a maximum, and the third neither (see also Exercise 16).

For $n = 2$, we do not even need to compute the eigenvalues, since $\lambda_1 \lambda_2 = \det(H)$ and $\lambda_1 + \lambda_2 = \operatorname{tr}(H)$ (Lecture 1). If $\det(H) < 0$, the eigenvalues have opposite signs, and the critical point is a saddle point. If $\det(H) > 0$, they have the same sign, which is the sign of $\operatorname{tr}(H)$: the critical point is a strict local minimum if $\operatorname{tr}(H) > 0$, and a strict local maximum if $\operatorname{tr}(H) < 0$.

eigenvalues of $\nabla^2 f(\mathbf{x}^*)$	graph near $\mathbf{x}^*$	level curves	$\mathbf{x}^*$ is a
all positive	bowl	ellipses	strict local minimum
all negative	upside-down bowl	ellipses	strict local maximum
positive and negative	saddle	hyperbolas	saddle point
some zero, the rest of one sign	trough	parallel lines	(test inconclusive)

Table 3: The second-order test at a critical point $\mathbf{x}^*$. The shape of the graph and the level curves (for $n = 2$) are those of the quadratic approximation $f(\mathbf{x}^*) + \frac{1}{2}(\lambda_1 y_1^2 + \cdots + \lambda_n y_n^2)$.

The eigenvalues also describe the level sets near a strict local minimum. For $c > 0$, the level set $\frac{1}{2}(\lambda_1 y_1^2 + \cdots + \lambda_n y_n^2) = c$ of the quadratic approximation is an ellipse (for $n = 2$) or an ellipsoid,

whose axes point in the directions of the eigenvectors $\mathbf{q}_i$ and have the half-lengths $\sqrt{2c/\lambda_i}$, as in the section on the spectral theorem in Lecture 2. So the level curves around a minimum are approximately ellipses that are long in directions of small curvature and short in directions of large curvature. The ratio of the longest and the shortest axis is $\sqrt{\lambda_1/\lambda_n}$, the square root of the condition number of the Hessian, and we will see in Section 10 that this elongation determines how fast gradient descent converges. Around a saddle point in the plane, the level curves of the quadratic approximation are hyperbolas, as for $x^2 - y^2$ in Figure 1.

Example 9.4. Let $f(x, y) = x^3 + y^3 - 3xy$. Its gradient is $\nabla f(x, y) = (3x^2 - 3y,\ 3y^2 - 3x)^\top$, so at a critical point, $y = x^2$ and $x = y^2 = x^4$. Hence $x = 0$ or $x^3 = 1$, i.e., $x = 1$, and the critical points are $(0, 0)$ and $(1, 1)$. The Hessian is

$$\nabla^2 f(x, y) = \begin{pmatrix} 6x & -3 \\ -3 & 6y \end{pmatrix}.$$

- At $(1, 1)$, the Hessian $\begin{pmatrix} 6 & -3 \\ -3 & 6 \end{pmatrix}$ has the eigenvector $(1, 1)^\top$ with eigenvalue 3, and the eigenvector $(1, -1)^\top$ with eigenvalue 9. Both eigenvalues are positive, so $(1, 1)$ is a strict local minimum, with $f(1, 1) = -1$. The curvature across the diagonal is three times as large as the curvature along it, so the level curves around $(1, 1)$ are approximately ellipses that are elongated in the direction of the diagonal, with axes in the ratio $\sqrt{9/3} = \sqrt{3}$ (Figure 7). The minimum is not global, since $f(x, 0) = x^3 \to -\infty$ as $x \to -\infty$.
- At $(0, 0)$, the Hessian $\begin{pmatrix} 0 & -3 \\ -3 & 0 \end{pmatrix}$ has the eigenvector $(1, 1)^\top$ with eigenvalue -3, and the eigenvector $(1, -1)^\top$ with eigenvalue 3. So the origin is a saddle point: f decreases along the diagonal $y = x$ and increases along the diagonal $y = -x$. Indeed, $f(t, t) = 2t^3 - 3t^2 \approx -3t^2$ and $f(t, -t) = 3t^2$ for small t, which are the parabolas $\frac{1}{2}\lambda t^2 \|\mathbf{v}\|^2$ for $\mathbf{v} = (1, \pm 1)^\top$. Note that both diagonal entries of the Hessian are 0: along the coordinate axes, $f(t, 0) = f(0, t) = t^3$ has no curvature at $t = 0$, and we would not see the saddle at all. As in Example 1.8, looking along the coordinate axes is not enough; the eigenvectors show the directions in which f really curves.

The level set $f = 0$ is the folium of Descartes $x^3 + y^3 = 3xy$, which crosses itself at the saddle point. Near the origin, it looks like the zero set of the quadratic approximation $\frac{1}{2}\mathbf{h}^\top H \mathbf{h} = -3xy$, which consists of the two coordinate axes.

Example 9.5 (Quadratic functions)**.** Let $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top A\mathbf{x} - \mathbf{b}^\top \mathbf{x}$, where A is symmetric positive definite. Then $\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$, so the only critical point is $\mathbf{x}^* = A^{-1}\mathbf{b}$, and the Hessian A is positive definite. In fact, $\mathbf{x}^*$ is the unique global minimum: using $\mathbf{b} = A\mathbf{x}^*$, one checks that

$$f(\mathbf{x}) - f(\mathbf{x}^*) = \tfrac{1}{2}(\mathbf{x} - \mathbf{x}^*)^\top A(\mathbf{x} - \mathbf{x}^*),$$

which is positive for $\mathbf{x} \neq \mathbf{x}^*$. The level sets of f are ellipsoids centered at $\mathbf{x}^*$, whose axes point in the directions of the eigenvectors of A (Lecture 2). So solving the linear system $A\mathbf{x} = \mathbf{b}$ with a symmetric positive definite matrix is the same as minimizing a quadratic function; this is the starting point of iterative methods for large linear systems, such as the conjugate gradient method. For example, the least squares loss $\|\mathbf{y} - \Phi\boldsymbol{\theta}\|^2 = \boldsymbol{\theta}^\top \Phi^\top \Phi \boldsymbol{\theta} - 2\mathbf{y}^\top \Phi \boldsymbol{\theta} + \mathbf{y}^\top \mathbf{y}$ is of this form with $A = 2\Phi^\top \Phi$ and $\mathbf{b} = 2\Phi^\top \mathbf{y}$, plus a constant. If Φ has linearly independent columns, then $\Phi^\top \Phi$ is positive definite (since $\boldsymbol{\theta}^\top \Phi^\top \Phi \boldsymbol{\theta} = \|\Phi\boldsymbol{\theta}\|^2 > 0$ for $\boldsymbol{\theta} \neq \mathbf{0}$), and the unique minimum is $\boldsymbol{\theta}^* = (\Phi^\top \Phi)^{-1}\Phi^\top \mathbf{y}$, as in Lecture 2.

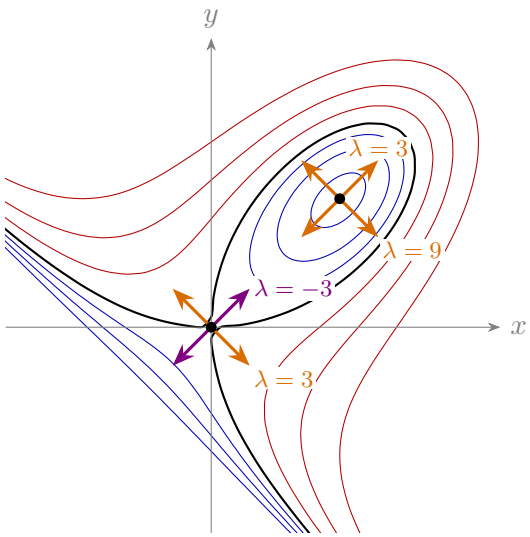


Figure 7: Level curves of $f(x, y) = x^3 + y^3 - 3xy$ for the levels $-0.9, -0.6, -0.3$ (blue), 0 (black: the folium of Descartes) and $0.6, 1.5, 3$ (red), and the eigenvectors of the Hessian at the two critical points (orange: positive eigenvalue, violet: negative eigenvalue). Around the minimum $(1, 1)$, the level curves are approximately ellipses with axes along the eigenvectors, longer in the direction of the smaller eigenvalue. At the saddle point $(0, 0)$, f decreases in the violet direction and increases in the orange one.

10 Gradient Descent and Newton's Method

For most models, such as logistic regression and neural networks, the equation $\nabla f(\mathbf{x}) = \mathbf{0}$ cannot be solved by a formula. Instead, we start with a guess $\mathbf{x}_0$ and improve it step by step.

Definition 10.1 (Gradient descent)**.** Let $f : \mathbb{R}^n \to \mathbb{R}$ be differentiable, let $\mathbf{x}_0 \in \mathbb{R}^n$, and let $\eta > 0$. Gradient descent with step size (or learning rate) η computes

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta \nabla f(\mathbf{x}_k), \qquad k = 0, 1, 2, \ldots$$

By Theorem 3.9, $-\nabla f(\mathbf{x}_k)$ is the direction of steepest descent, and by the linear approximation,

$$f(\mathbf{x}_k - \eta \nabla f(\mathbf{x}_k)) = f(\mathbf{x}_k) - \eta \|\nabla f(\mathbf{x}_k)\|^2 + o(\eta),$$

so a small enough step decreases f, unless $\nabla f(\mathbf{x}_k) = \mathbf{0}$. But how small is small enough? If η is too small, we make little progress; if it is too large, the linear approximation is no longer valid, and f can increase. The answer depends on how fast the gradient changes.

10.1 Convergence for smooth functions

Definition 10.2 (L-smooth function)**.** A differentiable function $f : \mathbb{R}^n \to \mathbb{R}$ is L-smooth, for a number $L > 0$, if its gradient is Lipschitz continuous with constant L:

$$\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \le L \|\mathbf{x} - \mathbf{y}\| \qquad \text{for all } \mathbf{x}, \mathbf{y} \in \mathbb{R}^n.$$

For example, the quadratic function $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top A \mathbf{x} - \mathbf{b}^\top \mathbf{x}$ with a symmetric matrix A has $\nabla f(\mathbf{x}) - \nabla f(\mathbf{y}) = A(\mathbf{x} - \mathbf{y})$, so it is L-smooth with $L = \|A\|_2$, which is the largest absolute value of an eigenvalue of A. More generally, one can show that a twice continuously differentiable function is L-smooth if and only if $\|\nabla^2 f(\mathbf{x})\|_2 \le L$ for all $\mathbf{x}$: the curvature is bounded by L.

Lemma 10.3 (Descent lemma)**.** *If f is L-smooth, then for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$,*

$$f(\mathbf{y}) \le f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2}\|\mathbf{y} - \mathbf{x}\|^2.$$

Proof. Let $\mathbf{d} = \mathbf{y} - \mathbf{x}$ and $g(t) = f(\mathbf{x} + t\mathbf{d})$. By the chain rule, $g'(t) = \nabla f(\mathbf{x} + t\mathbf{d})^\top \mathbf{d}$, which is continuous in t, since ∇f is Lipschitz continuous. By the fundamental theorem of calculus,

$$f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 (g'(t) - g'(0))\,dt = \int_0^1 \big(\nabla f(\mathbf{x} + t\mathbf{d}) - \nabla f(\mathbf{x})\big)^\top \mathbf{d}\,dt.$$

By the Cauchy–Schwarz inequality and L-smoothness, the integrand is at most $\|\nabla f(\mathbf{x} + t\mathbf{d}) - \nabla f(\mathbf{x})\|\|\mathbf{d}\| \le Lt\|\mathbf{d}\|^2$, and $\int_0^1 Lt\|\mathbf{d}\|^2\,dt = \frac{L}{2}\|\mathbf{d}\|^2$. □

So an L-smooth function lies below the quadratic function on the right, which touches it at $\mathbf{x}$. Gradient descent with step size $\frac{1}{L}$ minimizes this quadratic upper bound: the right-hand side is a quadratic function of $\mathbf{y}$ with gradient $\nabla f(\mathbf{x}) + L(\mathbf{y} - \mathbf{x})$, which is zero for $\mathbf{y} = \mathbf{x} - \frac{1}{L}\nabla f(\mathbf{x})$.

Theorem 10.4. *Let f be L-smooth and bounded below, $f(\mathbf{x}) \ge f^*$ for all $\mathbf{x}$, and let $0 < \eta \le \frac{1}{L}$. Then gradient descent satisfies*

$$f(\mathbf{x}_{k+1}) \le f(\mathbf{x}_k) - \frac{\eta}{2}\|\nabla f(\mathbf{x}_k)\|^2 \qquad \textit{and} \qquad \min_{0 \le k < K} \|\nabla f(\mathbf{x}_k)\|^2 \le \frac{2\big(f(\mathbf{x}_0) - f^*\big)}{\eta K}.$$

Proof. By the descent lemma with $\mathbf{x} = \mathbf{x}_k$ and $\mathbf{y} = \mathbf{x}_{k+1} = \mathbf{x}_k - \eta\nabla f(\mathbf{x}_k)$,

$$f(\mathbf{x}_{k+1}) \le f(\mathbf{x}_k) - \eta\|\nabla f(\mathbf{x}_k)\|^2 + \frac{L\eta^2}{2}\|\nabla f(\mathbf{x}_k)\|^2 = f(\mathbf{x}_k) - \eta\Big(1 - \frac{L\eta}{2}\Big)\|\nabla f(\mathbf{x}_k)\|^2,$$

and $1 - \frac{L\eta}{2} \ge \frac{1}{2}$ since $L\eta \le 1$. Adding these inequalities for $k = 0, \dots, K-1$ gives

$$\frac{\eta}{2}\sum_{k=0}^{K-1} \|\nabla f(\mathbf{x}_k)\|^2 \le f(\mathbf{x}_0) - f(\mathbf{x}_K) \le f(\mathbf{x}_0) - f^*,$$

and the minimum of the K numbers $\|\nabla f(\mathbf{x}_k)\|^2$ is at most their average. □

So with a small enough step size, gradient descent decreases f in every step, and it finds points where the gradient is small. It does not guarantee that we find a global minimum, or even a local minimum: it can approach a saddle point, or a local minimum that is much worse than the global one. For the non-convex loss functions of neural networks, this is essentially the best general guarantee. For convex functions (Section 11), the situation is much better.

10.2 Gradient descent on quadratic functions

Quadratic functions show precisely how the speed of gradient descent depends on the shape of the function. They are also important in general, since every twice continuously differentiable function is approximately quadratic near a minimum, with the eigenvalues of the Hessian as the curvatures along its principal directions (Section 9.1).

Let $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top A\mathbf{x} - \mathbf{b}^\top \mathbf{x}$, where A is symmetric positive definite with eigenvalues $\lambda_{\max} = \lambda_1 \ge \cdots \ge \lambda_n = \lambda_{\min} > 0$ and orthonormal eigenvectors $\mathbf{q}_1, \dots, \mathbf{q}_n$. By Example 9.5, the minimum is $\mathbf{x}^* = A^{-1}\mathbf{b}$, and $\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b} = A(\mathbf{x} - \mathbf{x}^*)$. So the error $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}^*$ satisfies

$$\mathbf{e}_{k+1} = \mathbf{e}_k - \eta A\mathbf{e}_k = (I - \eta A)\mathbf{e}_k, \qquad \text{hence} \qquad \mathbf{e}_k = (I - \eta A)^k \mathbf{e}_0.$$

The matrix $I - \eta A$ has the same eigenvectors as A, with the eigenvalues $1 - \eta\lambda_i$. So if we write $\mathbf{e}_0 = \sum_i c_i \mathbf{q}_i$, then

$$\mathbf{e}_k = \sum_{i=1}^{n} (1 - \eta\lambda_i)^k c_i \mathbf{q}_i :$$

in each eigenvector direction, the error is multiplied by $1 - \eta\lambda_i$ in every step.

Theorem 10.5. *Let f be as above, and let $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ be the condition number of A.*

1. $\|\mathbf{e}_k\| \le \rho^k \|\mathbf{e}_0\|$, *where* $\rho = \max_i |1 - \eta\lambda_i| = \max\left(|1 - \eta\lambda_{\max}|, |1 - \eta\lambda_{\min}|\right)$.
2. *Gradient descent converges to $\mathbf{x}^*$ for every $\mathbf{x}_0$ if and only if $0 < \eta < \frac{2}{\lambda_{\max}}$.*
3. *The rate ρ is smallest for $\eta = \frac{2}{\lambda_{\max} + \lambda_{\min}}$, and then $\rho = \frac{\kappa - 1}{\kappa + 1}$.*

Proof. (1) Since the $\mathbf{q}_i$ are orthonormal, $\|\mathbf{e}_k\|^2 = \sum_i (1 - \eta\lambda_i)^{2k} c_i^2 \le \rho^{2k} \sum_i c_i^2 = \rho^{2k} \|\mathbf{e}_0\|^2$. The maximum of $|1 - \eta\lambda|$ over $\lambda \in [\lambda_{\min}, \lambda_{\max}]$ is attained at one of the endpoints.

(2) If $0 < \eta < \frac{2}{\lambda_{\max}}$, then $-1 < 1 - \eta\lambda_i < 1$ for all i, so $\rho < 1$ and $\mathbf{e}_k \to \mathbf{0}$ by (1). Conversely, if $\eta \ge \frac{2}{\lambda_{\max}}$, then $1 - \eta\lambda_1 \le -1$, and for $\mathbf{x}_0 = \mathbf{x}^* + \mathbf{q}_1$, the error $\mathbf{e}_k = (1 - \eta\lambda_1)^k \mathbf{q}_1$ does not tend to $\mathbf{0}$.

(3) Let $\eta^* = \frac{2}{\lambda_{\max} + \lambda_{\min}}$. Then $1 - \eta^*\lambda_{\min} = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa - 1}{\kappa + 1}$ and $1 - \eta^*\lambda_{\max} = -\frac{\kappa - 1}{\kappa + 1}$, so $\rho(\eta^*) = \frac{\kappa - 1}{\kappa + 1}$. For $\eta < \eta^*$, we have $|1 - \eta\lambda_{\min}| \ge 1 - \eta\lambda_{\min} > 1 - \eta^*\lambda_{\min}$, and for $\eta > \eta^*$, we have $|1 - \eta\lambda_{\max}| \ge \eta\lambda_{\max} - 1 > \eta^*\lambda_{\max} - 1$. In both cases, $\rho(\eta) > \rho(\eta^*)$. □

Example 10.6. Let $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 10x_2^2)$, i.e., $A = \mathrm{diag}(1, 10)$, $\mathbf{b} = \mathbf{0}$ and $\mathbf{x}^* = \mathbf{0}$. Gradient descent multiplies x_1 by $1 - \eta$ and x_2 by $1 - 10\eta$ in every step. It converges if and only if $\eta < 0.2$; for $\eta = 0.21$, for example, $|x_2|$ grows by the factor 1.1 in every step. The best step size is $\eta = \frac{2}{11}$, with $\rho = \frac{9}{11} \approx 0.82$: then x_1 is multiplied by $\frac{9}{11}$, and x_2 by $-\frac{9}{11}$, so the iterates jump back and forth across the valley, while they slowly move along it (Figure 8). To reduce the error by a factor 10^{-6}, we need k steps with $(\frac{9}{11})^k \le 10^{-6}$, i.e., $k \ge 69$. For $A = I$, the step size $\eta = 1$ reaches the minimum in a single step.

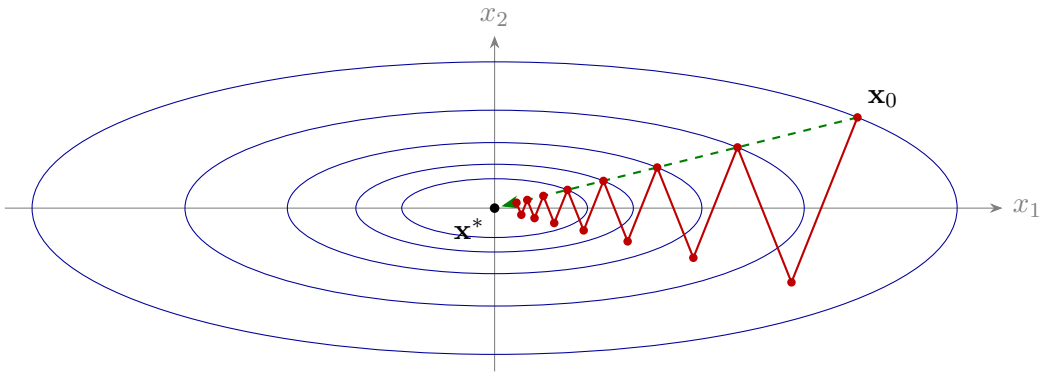


Figure 8: Gradient descent on $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 10x_2^2)$ with the optimal step size $\eta = \frac{2}{11}$, starting at $\mathbf{x}_0 = (8, 2)^\top$ (red: the first 14 steps). The ellipses are level curves of f. Since the condition number is $\kappa = 10$, the level curves are elongated, the gradient points mostly across the valley instead of towards the minimum, and the iterates zig-zag. Newton's method (dashed, Section 10.3) reaches the minimum in one step.

For large κ, the rate $\frac{\kappa - 1}{\kappa + 1} \approx 1 - \frac{2}{\kappa}$ is close to 1, and the number of steps needed for a given accuracy grows proportionally to κ. Ill-conditioned problems, whose level sets are long and thin

ellipsoids, are slow for gradient descent. This is the zig-zagging that was mentioned at the end of the section on the spectral theorem in Lecture 2.

Remark 10.7 (Making gradient descent faster).

1. Feature scaling. For least squares, the Hessian is $2\Phi^\top\Phi$. If the features (the columns of Φ) are measured on very different scales, then κ is large. Standardizing each feature, i.e., subtracting its mean and dividing by its standard deviation, often reduces κ dramatically.

2. Momentum. The heavy ball method $\mathbf{x}_{k+1} = \mathbf{x}_k - \eta\nabla f(\mathbf{x}_k) + \beta(\mathbf{x}_k - \mathbf{x}_{k-1})$ keeps some of the previous step. This damps the zig-zagging and accelerates the motion along the valley. For quadratic functions, with well-chosen η and β, the error decreases roughly like $\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k$, so the number of steps grows only like $\sqrt{\kappa}$.

3. Preconditioning. Gradient descent with respect to a different inner product $\langle \mathbf{x}, \mathbf{y}\rangle_P = \mathbf{x}^\top P\mathbf{y}$ uses the step $-\eta P^{-1}\nabla f(\mathbf{x})$ (Remark 3.8), which is the same as gradient descent after a linear change of variables. A good choice of P makes the level sets rounder. Optimizers such as Adam use a diagonal matrix P that is adapted during training. The extreme case $P = \nabla^2 f(\mathbf{x})$ is Newton's method.

Remark 10.8 (Stochastic gradient descent). In machine learning, the loss is an average over the training data, $L(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \ell_i(\boldsymbol{\theta})$, where ℓ_i is the loss on the i-th data point. Computing the full gradient needs a pass over all N data points, which is expensive for large data sets. Stochastic gradient descent (SGD) instead chooses a small random subset $B \subseteq \{1, \dots, N\}$ (a mini-batch) in every step and uses the mini-batch gradient

$$\mathbf{g} = \frac{1}{|B|}\sum_{i\in B} \nabla\ell_i(\boldsymbol{\theta}),$$

which is cheap to compute. If B is chosen uniformly at random, then the expected value of $\mathbf{g}$ is the full gradient $\nabla L(\boldsymbol{\theta})$, so on average SGD moves in the right direction. The noise in $\mathbf{g}$ means that SGD does not settle exactly at a minimum unless the step size is decreased during training. In practice, SGD and its variants with momentum and preconditioning are used to train almost all neural networks.

10.3 Newton's method

Gradient descent only uses the first-order approximation of f. Newton's method uses the second-order approximation: at $\mathbf{x}_k$, it minimizes the quadratic model

$$m_k(\mathbf{h}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top\mathbf{h} + \tfrac{1}{2}\mathbf{h}^\top\nabla^2 f(\mathbf{x}_k)\mathbf{h}$$

of Theorem 8.5. If the Hessian $H_k = \nabla^2 f(\mathbf{x}_k)$ is positive definite, then by Example 9.5 (with $A = H_k$ and $\mathbf{b} = -\nabla f(\mathbf{x}_k)$), the minimum of m_k is $\mathbf{h} = -H_k^{-1}\nabla f(\mathbf{x}_k)$. This gives Newton's method:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \nabla^2 f(\mathbf{x}_k)^{-1}\nabla f(\mathbf{x}_k).$$

In practice, we do not invert the Hessian, but solve the linear system $H_k\mathbf{h} = -\nabla f(\mathbf{x}_k)$ (Lecture 1). For $n = 1$, Newton's method is $x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$, i.e., Newton's method for finding a zero of f'.

- If f is a quadratic function with a positive definite matrix A, then $m_0 = f$, and Newton's method finds the minimum in a single step, no matter how large the condition number is (Figure 8).

- Near a strict local minimum $\mathbf{x}^*$ with a positive definite Hessian, Newton's method converges quadratically: roughly, the error is squared in every step, $\|\mathbf{x}_{k+1} - \mathbf{x}^*\| \le C\|\mathbf{x}_k - \mathbf{x}^*\|^2$, so the number of correct digits doubles in every step. We do not prove this.

Example 10.9. Let $f(x) = e^x - 2x$. Then $f'(x) = e^x - 2$ and $f''(x) = e^x > 0$, so the minimum is at $x^* = \log 2 = 0.693147\ldots$, and Newton's method is $x_{k+1} = x_k - \frac{e^{x_k}-2}{e^{x_k}} = x_k - 1 + 2e^{-x_k}$. Starting at $x_0 = 0$:

k	0	1	2	3	4	5
x_k	0	1	0.735759	0.694042	0.6931476	0.693147180560
$\lvert x_k - x^*\rvert$	0.69	0.31	$4.3 \cdot 10^{-2}$	$9.0 \cdot 10^{-4}$	$4.0 \cdot 10^{-7}$	$8.0 \cdot 10^{-14}$

Once the iterates are close to x^*, the exponent of the error roughly doubles in every step.

Newton's method has two drawbacks. First, for n parameters, the Hessian has n^2 entries, and solving the linear system costs about n^3 operations, which is impossible for neural networks with millions of parameters. Second, far from a minimum, the Hessian need not be positive definite, and Newton's method can move to a saddle point or a maximum, since it only looks for a critical point. Quasi-Newton methods such as BFGS and L-BFGS build approximations of the Hessian (or of its inverse) from the gradients of previous steps, and are popular for problems with up to a few thousand parameters. Note also that the Newton step $-H_k^{-1}\nabla f(\mathbf{x}_k)$ is the gradient step with respect to the inner product $\langle \mathbf{x}, \mathbf{y}\rangle_{H_k} = \mathbf{x}^\top H_k \mathbf{y}$ defined by the Hessian (Remark 3.8): Newton's method measures distances in a way that makes the level sets of the quadratic model round.

11 Convexity

Gradient descent and Newton's method only find critical points, and a critical point need not be a global minimum. For an important class of functions, the convex functions, this problem disappears.

Definition 11.1 (Convex sets and functions)**.** A set $C \subseteq \mathbb{R}^n$ is convex if for all $\mathbf{x}, \mathbf{y} \in C$ and all $\theta \in [0,1]$, also $\theta\mathbf{x} + (1-\theta)\mathbf{y} \in C$: with any two points, C contains the line segment between them. A function $f : C \to \mathbb{R}$ on a convex set C is convex if

$$f(\theta\mathbf{x} + (1-\theta)\mathbf{y}) \le \theta f(\mathbf{x}) + (1-\theta) f(\mathbf{y}) \qquad \text{for all } \mathbf{x}, \mathbf{y} \in C,\ \theta \in [0,1],$$

and strictly convex if the inequality is strict whenever $\mathbf{x} \ne \mathbf{y}$ and $0 < \theta < 1$. A function f is concave if $-f$ is convex.

Geometrically, f is convex if the line segment between any two points of its graph lies above the graph (Figure 9). Examples of convex sets are $\mathbb{R}^n$, subspaces, affine subspaces, half-spaces $\{\mathbf{x} : \mathbf{w}^\top\mathbf{x} \le c\}$, balls $\{\mathbf{x} : \|\mathbf{x} - \mathbf{a}\| \le r\}$ for any norm, and intersections of convex sets. A circle is not convex.

Example 11.2.

1. Affine functions $f(\mathbf{x}) = \mathbf{a}^\top\mathbf{x} + b$ are convex (and concave), with equality in the definition.
2. Every norm is convex: by the triangle inequality and homogeneity, $\|\theta\mathbf{x} + (1-\theta)\mathbf{y}\| \le \theta\|\mathbf{x}\| + (1-\theta)\|\mathbf{y}\|$ for $\theta \in [0,1]$.
3. The functions x^2, $|x|$, e^x on $\mathbb{R}$, and $-\log x$ on $(0,\infty)$ are convex. The function x^3 is not convex on $\mathbb{R}$.

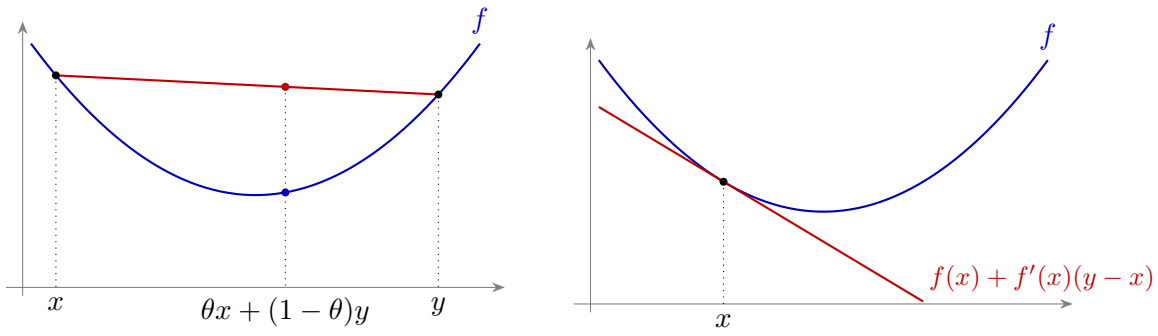


Figure 9: Left: a convex function lies below every chord, i.e., $f(\theta x+(1-\theta)y) \le \theta f(x)+(1-\theta)f(y)$. Right: a differentiable convex function lies above every tangent line (Theorem 11.3).

Theorem 11.3 (First-order characterization)**.** *Let $C \subseteq \mathbb{R}^n$ be open and convex, and let $f : C \to \mathbb{R}$ be differentiable. Then f is convex if and only if*

$$f(\mathbf{y}) \ge f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) \qquad \text{for all } \mathbf{x}, \mathbf{y} \in C.$$

Proof. Suppose that f is convex, and let $\mathbf{x}, \mathbf{y} \in C$. For $0 < t \le 1$, convexity with $\theta = 1 - t$ gives $f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) \le f(\mathbf{x}) + t(f(\mathbf{y}) - f(\mathbf{x}))$, so

$$\frac{f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) - f(\mathbf{x})}{t} \le f(\mathbf{y}) - f(\mathbf{x}).$$

As $t \to 0$, the left side tends to the directional derivative $\nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x})$ (Proposition 3.5).

Conversely, suppose that the inequality holds, let $\theta \in [0, 1]$, and let $\mathbf{z} = \theta\mathbf{x} + (1 - \theta)\mathbf{y}$. Then $f(\mathbf{x}) \ge f(\mathbf{z}) + \nabla f(\mathbf{z})^\top (\mathbf{x} - \mathbf{z})$ and $f(\mathbf{y}) \ge f(\mathbf{z}) + \nabla f(\mathbf{z})^\top (\mathbf{y} - \mathbf{z})$. Multiplying the first inequality by θ and the second by $1 - \theta$ and adding, we get

$$\theta f(\mathbf{x}) + (1 - \theta) f(\mathbf{y}) \ge f(\mathbf{z}) + \nabla f(\mathbf{z})^\top \big(\theta\mathbf{x} + (1 - \theta)\mathbf{y} - \mathbf{z}\big) = f(\mathbf{z}). \qquad \square$$

So the linear approximation of a convex function is a global lower bound. This has a remarkable consequence.

Corollary 11.4. *Let f be convex and differentiable on an open convex set C. If $\nabla f(\mathbf{x}^*) = \mathbf{0}$, then $\mathbf{x}^*$ is a global minimum of f.*

Proof. By Theorem 11.3, $f(\mathbf{y}) \ge f(\mathbf{x}^*) + \mathbf{0}^\top (\mathbf{y} - \mathbf{x}^*) = f(\mathbf{x}^*)$ for all $\mathbf{y} \in C$. $\square$

Proposition 11.5. *Let $f : C \to \mathbb{R}$ be convex. Then every local minimum of f is a global minimum, and the set of global minima is convex. If f is strictly convex, it has at most one global minimum.*

Proof. Let $\mathbf{x}^*$ be a local minimum, and suppose that $f(\mathbf{y}) < f(\mathbf{x}^*)$ for some $\mathbf{y} \in C$. For $0 < t \le 1$, the point $\mathbf{x}_t = (1 - t)\mathbf{x}^* + t\mathbf{y}$ satisfies $f(\mathbf{x}_t) \le (1 - t)f(\mathbf{x}^*) + tf(\mathbf{y}) < f(\mathbf{x}^*)$, and $\mathbf{x}_t$ is arbitrarily close to $\mathbf{x}^*$ for small t. This contradicts the local minimality of $\mathbf{x}^*$. If $\mathbf{x}$ and $\mathbf{y}$ are global minima with $f(\mathbf{x}) = f(\mathbf{y}) = m$, then $f(\theta\mathbf{x} + (1 - \theta)\mathbf{y}) \le m$, so all points of the segment are global minima. For strictly convex f and $\mathbf{x} \ne \mathbf{y}$, we would get $f(\frac{1}{2}\mathbf{x} + \frac{1}{2}\mathbf{y}) < m$, which is impossible. $\square$

So for convex functions, there are no bad local minima and no saddle points: every critical point is a global minimum, and gradient descent with a suitable step size converges to the minimum value. (One can show that for a convex L-smooth function with a minimum $\mathbf{x}^*$, gradient descent with $\eta = \frac{1}{L}$ satisfies $f(\mathbf{x}_k) - f(\mathbf{x}^*) \le \frac{L\|\mathbf{x}_0 - \mathbf{x}^*\|^2}{2k}$.) For twice differentiable functions, convexity can be checked with the Hessian.

Theorem 11.6 (Second-order characterization)**.** *Let $C \subseteq \mathbb{R}^n$ be open and convex, and let $f : C \to \mathbb{R}$ be twice continuously differentiable. Then f is convex if and only if $\nabla^2 f(\mathbf{x})$ is positive semidefinite for all $\mathbf{x} \in C$. If $\nabla^2 f(\mathbf{x})$ is positive definite for all $\mathbf{x} \in C$, then f is strictly convex.*

Proof. If all Hessians are positive semidefinite, then by Theorem 8.5, for all $\mathbf{x}, \mathbf{y} \in C$,

$$f(\mathbf{y}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top(\mathbf{y}-\mathbf{x}) + \tfrac{1}{2}(\mathbf{y}-\mathbf{x})^\top \nabla^2 f(\mathbf{z})(\mathbf{y}-\mathbf{x}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top(\mathbf{y}-\mathbf{x}),$$

where $\mathbf{z}$ is a point on the segment between $\mathbf{x}$ and $\mathbf{y}$, so f is convex by Theorem 11.3. (If all Hessians are positive definite, the inequality is strict for $\mathbf{y} \neq \mathbf{x}$, and the proof of Theorem 11.3 then gives strict convexity.) Conversely, suppose that $\mathbf{v}^\top \nabla^2 f(\mathbf{x})\mathbf{v} < 0$ for some $\mathbf{x} \in C$ and $\mathbf{v}$. By the second form of Theorem 8.5, for small $t \neq 0$,

$$f(\mathbf{x}+t\mathbf{v}) = f(\mathbf{x}) + t\nabla f(\mathbf{x})^\top \mathbf{v} + \tfrac{1}{2}t^2 \mathbf{v}^\top \nabla^2 f(\mathbf{x})\mathbf{v} + o(t^2) < f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (t\mathbf{v}),$$

which contradicts the inequality of Theorem 11.3 for $\mathbf{y} = \mathbf{x} + t\mathbf{v}$. □

Example 11.7 (Convex loss functions)**.**

1. The least squares loss $\|\mathbf{y} - \Phi\boldsymbol{\theta}\|^2$ has the Hessian $2\Phi^\top\Phi$, which is positive semidefinite, since $\mathbf{v}^\top\Phi^\top\Phi\mathbf{v} = \|\Phi\mathbf{v}\|^2 \geq 0$. So it is convex, and strictly convex if Φ has linearly independent columns. Adding a ridge penalty $\lambda\|\boldsymbol{\theta}\|^2$ with $\lambda > 0$ adds $2\lambda I$ to the Hessian, which makes it positive definite, so ridge regression always has a unique minimum.

2. The logistic regression loss has the Hessian $\frac{1}{N}\sum_i p_i(1-p_i)\mathbf{x}_i\mathbf{x}_i^\top$ (Example 8.4). It is positive semidefinite, since $\mathbf{v}^\top\mathbf{x}_i\mathbf{x}_i^\top\mathbf{v} = (\mathbf{x}_i^\top\mathbf{v})^2 \geq 0$ and $p_i(1-p_i) > 0$. So the loss is convex, and every critical point is a global minimum.

3. The log-sum-exp function has the Hessian $\operatorname{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$, where $\mathbf{p}$ has positive entries with sum 1. For every $\mathbf{v}$, by the Cauchy–Schwarz inequality applied to the vectors with entries $\sqrt{p_i}$ and $\sqrt{p_i}v_i$,
$$\Big(\sum_i p_i v_i\Big)^2 \leq \Big(\sum_i p_i\Big)\Big(\sum_i p_i v_i^2\Big) = \sum_i p_i v_i^2,$$
so $\mathbf{v}^\top\big(\operatorname{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top\big)\mathbf{v} = \sum_i p_i v_i^2 - \big(\sum_i p_i v_i\big)^2 \geq 0$. So log-sum-exp is convex, and hence so is the softmax cross-entropy loss of Example 6.6.

Proposition 11.8 (Operations that preserve convexity)**.**

1. *If f and g are convex and $\alpha, \beta \geq 0$, then $\alpha f + \beta g$ is convex.*
2. *If f is convex, then $\mathbf{x} \mapsto f(A\mathbf{x} + \mathbf{b})$ is convex.*
3. *If f and g are convex, then $\mathbf{x} \mapsto \max(f(\mathbf{x}), g(\mathbf{x}))$ is convex.*

Proof. (1) Add the inequalities for f and g, multiplied by α and β. (2) The map $\mathbf{x} \mapsto A\mathbf{x}+\mathbf{b}$ maps $\theta\mathbf{x} + (1-\theta)\mathbf{y}$ to $\theta(A\mathbf{x}+\mathbf{b}) + (1-\theta)(A\mathbf{y}+\mathbf{b})$; now use the convexity of f. (3) $f(\theta\mathbf{x} + (1-\theta)\mathbf{y}) \leq \theta f(\mathbf{x}) + (1-\theta)f(\mathbf{y}) \leq \theta \max(f,g)(\mathbf{x}) + (1-\theta)\max(f,g)(\mathbf{y})$, and similarly for g. □

For example, the hinge loss $\max(0, 1 - y\,\mathbf{w}^\top\mathbf{x})$ of support vector machines is convex in $\mathbf{w}$, as the maximum of two affine functions, and so are the ℓ_1 and ℓ_2 regularization terms $\lambda\|\mathbf{w}\|_1$ and $\lambda\|\mathbf{w}\|^2$.

Remark 11.9 (Neural networks are not convex). The loss of a neural network with a hidden layer is not convex as a function of the weights. One reason is symmetry: if we swap two hidden units, i.e., swap two rows of W_1, the corresponding entries of $\mathbf{b}_1$, and the corresponding columns of W_2, the network computes the same function, so the loss does not change. If $\boldsymbol{\theta}^*$ is a global minimum, then so is the swapped parameter vector $\boldsymbol{\theta}'$. If the loss were convex, then by Proposition 11.5 the midpoint $\frac{1}{2}(\boldsymbol{\theta}^* + \boldsymbol{\theta}')$ would be a global minimum too. But in the midpoint, the two hidden units have identical weights, so the network effectively has one hidden unit less, and its loss is usually larger. Nevertheless, gradient-based methods work remarkably well for neural networks in practice, which is an active area of research.

12 Constrained Optimization

Many optimization problems in machine learning have constraints. In principal component analysis, we maximize the variance over unit vectors; a vector of probabilities must have nonnegative entries that sum to 1; and support vector machines maximize a margin subject to classifying all training points correctly. We first consider equality constraints:

$$\text{minimize } f(\mathbf{x}) \quad \text{subject to} \quad g_1(\mathbf{x}) = 0,\ \ldots,\ g_m(\mathbf{x}) = 0.$$

12.1 Lagrange multipliers

Consider first one constraint $g(\mathbf{x}) = 0$ in $\mathbb{R}^2$, which is usually a curve. If we walk along the curve, then at a local minimum of f on the curve, the derivative of f in the direction of the curve must be zero, i.e., $\nabla f(\mathbf{x}^*)$ is orthogonal to the curve. By Corollary 5.4, $\nabla g(\mathbf{x}^*)$ is also orthogonal to the curve, which is a level set of g. In the plane, this means that $\nabla f(\mathbf{x}^*)$ and $\nabla g(\mathbf{x}^*)$ are parallel: the level curve of f through $\mathbf{x}^*$ touches the constraint curve (Figure 10). At a point where ∇f is not orthogonal to the curve, we can decrease f by moving along the curve.

Theorem 12.1 (Lagrange multipliers)**.** *Let $f, g_1, \ldots, g_m : D \to \mathbb{R}$ be continuously differentiable on an open set $D \subseteq \mathbb{R}^n$, and let $\mathbf{x}^*$ be a local minimum of f on the set $S = \{\mathbf{x} \in D : g_1(\mathbf{x}) = \cdots = g_m(\mathbf{x}) = 0\}$. If the gradients $\nabla g_1(\mathbf{x}^*), \ldots, \nabla g_m(\mathbf{x}^*)$ are linearly independent, then there are unique numbers $\lambda_1, \ldots, \lambda_m \in \mathbb{R}$, called Lagrange multipliers, such that*

$$\nabla f(\mathbf{x}^*) + \lambda_1 \nabla g_1(\mathbf{x}^*) + \cdots + \lambda_m \nabla g_m(\mathbf{x}^*) = \mathbf{0}.$$

Sketch of proof. Let $U = \operatorname{span}\{\nabla g_1(\mathbf{x}^*), \ldots, \nabla g_m(\mathbf{x}^*)\}$. If $\mathbf{x}(t)$ is a differentiable curve in S with $\mathbf{x}(0) = \mathbf{x}^*$, then $g_i(\mathbf{x}(t)) = 0$ for all t, so $\nabla g_i(\mathbf{x}^*)^\top \mathbf{x}'(0) = 0$ by the chain rule, i.e., the tangent vector $\mathbf{x}'(0)$ lies in $U^\perp$. Conversely, the implicit function theorem from analysis, which we do not prove, shows that, since the $\nabla g_i(\mathbf{x}^*)$ are linearly independent, every vector $\mathbf{v} \in U^\perp$ is the tangent vector $\mathbf{x}'(0)$ of such a curve. Since $t \mapsto f(\mathbf{x}(t))$ has a local minimum at $t = 0$, its derivative $\nabla f(\mathbf{x}^*)^\top \mathbf{x}'(0)$ is 0. Hence $\nabla f(\mathbf{x}^*)$ is orthogonal to every vector in $U^\perp$, i.e., $\nabla f(\mathbf{x}^*) \in (U^\perp)^\perp = U$ (Lecture 2). So $\nabla f(\mathbf{x}^*)$ is a linear combination of the $\nabla g_i(\mathbf{x}^*)$, and the coefficients are unique, since the $\nabla g_i(\mathbf{x}^*)$ are linearly independent. □

The same condition holds at local maxima (apply the theorem to $-f$; the multipliers can have any sign). It is convenient to write it with the Lagrangian

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \lambda_1 g_1(\mathbf{x}) + \cdots + \lambda_m g_m(\mathbf{x}).$$

The condition of the theorem says that $\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^*, \boldsymbol{\lambda}) = \mathbf{0}$, and the constraints say that $\nabla_{\boldsymbol{\lambda}} \mathcal{L}(\mathbf{x}^*, \boldsymbol{\lambda}) = (g_1(\mathbf{x}^*), \ldots, g_m(\mathbf{x}^*))^\top = \mathbf{0}$. So we look for critical points of $\mathcal{L}$, which gives $n + m$ equations for

the $n+m$ unknowns $\mathbf{x}$ and $\boldsymbol{\lambda}$. If S is closed and bounded (such as a sphere), then f attains its minimum and maximum on S (this is a theorem from analysis), and we find them by comparing the values of f at all solutions of these equations.

Example 12.2. We find the maximum and minimum of $f(x,y) = x+y$ on the unit circle $g(x,y) = x^2+y^2-1=0$. The condition $\nabla f + \lambda \nabla g = \mathbf{0}$ is $(1,1)+\lambda(2x,2y)=(0,0)$, so $x=y=-\frac{1}{2\lambda}$, and with $x^2+y^2=1$, we get $x=y=\pm\frac{1}{\sqrt{2}}$. The maximum is $f(\frac{1}{\sqrt{2}},\frac{1}{\sqrt{2}})=\sqrt{2}$, and the minimum is $-\sqrt{2}$. This agrees with the Cauchy–Schwarz inequality: $x+y=(1,1)(x,y)^\top \le \sqrt{2}\sqrt{x^2+y^2}=\sqrt{2}$. See Figure 10.

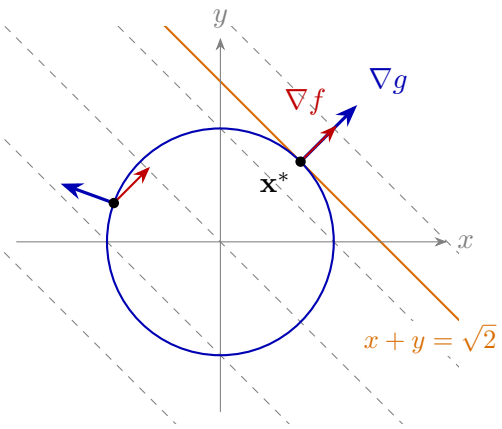


Figure 10: Maximizing $f(x,y)=x+y$ on the unit circle $g(x,y)=x^2+y^2-1=0$. The dashed lines are level lines of f. At the maximum $\mathbf{x}^*$, the level line $x+y=\sqrt{2}$ touches the circle, and ∇f and ∇g are parallel. At the other marked point, they are not parallel: ∇f has a component along the circle, so moving along the circle in that direction increases f.

Example 12.3 (Eigenvalues and PCA)**.** Let A be a symmetric $n \times n$ matrix. We maximize $f(\mathbf{x}) = \mathbf{x}^\top A\mathbf{x}$ subject to $g(\mathbf{x}) = 1-\mathbf{x}^\top\mathbf{x}=0$. Here $\nabla g(\mathbf{x}) = -2\mathbf{x} \neq \mathbf{0}$ on the unit sphere, and the Lagrange condition $2A\mathbf{x}-2\lambda\mathbf{x}=\mathbf{0}$ says that

$$A\mathbf{x} = \lambda\mathbf{x}\,:$$

the candidates are exactly the unit eigenvectors of A, and the Lagrange multiplier is the eigenvalue. At a unit eigenvector, $f(\mathbf{x}) = \mathbf{x}^\top(\lambda\mathbf{x}) = \lambda$. Since the sphere is closed and bounded, the maximum exists, and it is the largest eigenvalue $\lambda_{\max}$, attained at a corresponding eigenvector. This is the Rayleigh quotient result of Lecture 2, found with calculus instead of the spectral theorem; in PCA, it gives the first principal component.

Example 12.4 (The minimum norm solution)**.** Let $A \in \mathbb{R}^{m\times n}$ have linearly independent rows $\mathbf{a}_1^\top,\dots,\mathbf{a}_m^\top$ (so $m \le n$), and let $\mathbf{b}\in\mathbb{R}^m$. The system $A\mathbf{x}=\mathbf{b}$ has infinitely many solutions if $m<n$, and we look for the one with the smallest norm: minimize $\|\mathbf{x}\|^2$ subject to $g_i(\mathbf{x}) = \mathbf{a}_i^\top\mathbf{x}-b_i=0$ for $i=1,\dots,m$. The gradients $\nabla g_i = \mathbf{a}_i$ are linearly independent, and the Lagrange condition is

$$2\mathbf{x}+\sum_{i=1}^{m}\lambda_i\mathbf{a}_i = 2\mathbf{x}+A^\top\boldsymbol{\lambda}=\mathbf{0}, \qquad \text{so} \qquad \mathbf{x}=A^\top\boldsymbol{\mu} \text{ with } \boldsymbol{\mu}=-\tfrac{1}{2}\boldsymbol{\lambda}.$$

So $\mathbf{x}$ lies in the row space of A. Substituting into $A\mathbf{x} = \mathbf{b}$ gives $AA^\top \boldsymbol{\mu} = \mathbf{b}$. The $m \times m$ matrix $AA^\top$ is invertible (it is positive definite, since $\mathbf{v}^\top AA^\top \mathbf{v} = \|A^\top \mathbf{v}\|^2 > 0$ for $\mathbf{v} \neq \mathbf{0}$), so

$$\mathbf{x}^* = A^\top (AA^\top)^{-1} \mathbf{b}.$$

This is indeed the global minimum: every solution of $A\mathbf{x} = \mathbf{b}$ is $\mathbf{x} = \mathbf{x}^* + \mathbf{v}$ with $\mathbf{v} \in \operatorname{null}(A)$, and since $\mathbf{x}^* \in \operatorname{row}(A) = \operatorname{null}(A)^\perp$, Pythagoras gives $\|\mathbf{x}\|^2 = \|\mathbf{x}^*\|^2 + \|\mathbf{v}\|^2 \geq \|\mathbf{x}^*\|^2$. By the proposition on the pseudo-inverse in Lecture 2, applied to $A^\top$ (which has linearly independent columns), $(A^\top)^+ = (AA^\top)^{-1} A$. By the definition of the pseudo-inverse with the SVD, $(A^\top)^+ = (A^+)^\top$, so taking transposes gives $A^+ = A^\top (AA^\top)^{-1}$. So $\mathbf{x}^* = A^+ \mathbf{b}$, which proves, for matrices with linearly independent rows, the claim of Lecture 2 that $A^+ \mathbf{b}$ is the solution with the smallest norm.

12.2 Inequality constraints and the KKT conditions

Now consider inequality constraints:

$$\text{minimize } f(\mathbf{x}) \quad \text{subject to} \quad g_1(\mathbf{x}) \leq 0, \ \ldots, \ g_m(\mathbf{x}) \leq 0.$$

At a feasible point $\mathbf{x}^*$, the constraint g_i is active if $g_i(\mathbf{x}^*) = 0$, and inactive if $g_i(\mathbf{x}^*) < 0$. Inactive constraints do not matter near $\mathbf{x}^*$, since they remain satisfied when we move $\mathbf{x}^*$ a little. Active constraints behave like equality constraints, with one difference: moving into the feasible region, where $g_i < 0$, is allowed. At a minimum, this must not decrease f, and this forces the multipliers to be nonnegative.

Theorem 12.5 (Karush–Kuhn–Tucker conditions)**.** *Let $f, g_1, \ldots, g_m$ be continuously differentiable, and let $\mathbf{x}^*$ be a local minimum of f subject to $g_1(\mathbf{x}) \leq 0, \ldots, g_m(\mathbf{x}) \leq 0$, such that the gradients of the active constraints at $\mathbf{x}^*$ are linearly independent. Then there are numbers $\mu_1, \ldots, \mu_m$ such that:*

1. *(stationarity) $\nabla f(\mathbf{x}^*) + \mu_1 \nabla g_1(\mathbf{x}^*) + \cdots + \mu_m \nabla g_m(\mathbf{x}^*) = \mathbf{0}$;*
2. *(primal feasibility) $g_i(\mathbf{x}^*) \leq 0$ for all i;*
3. *(dual feasibility) $\mu_i \geq 0$ for all i;*
4. *(complementary slackness) $\mu_i g_i(\mathbf{x}^*) = 0$ for all i, i.e., $\mu_i = 0$ for every inactive constraint.*

Conversely, if f and all g_i are convex and differentiable, then every $\mathbf{x}^*$ that satisfies (1)–(4) for some $\mu_1, \ldots, \mu_m$ is a global minimum.

Proof of the converse. We do not prove the first part. For the converse, let $\mathbf{x}$ be feasible. The function $\mathcal{L}(\mathbf{z}) = f(\mathbf{z}) + \sum_i \mu_i g_i(\mathbf{z})$ is convex by Proposition 11.8, since $\mu_i \geq 0$, and $\nabla \mathcal{L}(\mathbf{x}^*) = \mathbf{0}$ by (1). By Corollary 11.4, $\mathcal{L}(\mathbf{x}) \geq \mathcal{L}(\mathbf{x}^*)$. Since $\mu_i g_i(\mathbf{x}) \leq 0$ for feasible $\mathbf{x}$ and $\mu_i g_i(\mathbf{x}^*) = 0$ by (4),

$$f(\mathbf{x}) \geq f(\mathbf{x}) + \sum_i \mu_i g_i(\mathbf{x}) = \mathcal{L}(\mathbf{x}) \geq \mathcal{L}(\mathbf{x}^*) = f(\mathbf{x}^*). \qquad \square$$

Equality constraints can be added to the KKT conditions with multipliers of any sign, as in Theorem 12.1.

Example 12.6. Minimize $f(x) = (x-2)^2$ subject to $x \leq 1$, i.e., $g(x) = x - 1 \leq 0$. The KKT conditions are $2(x-2) + \mu = 0$, $x \leq 1$, $\mu \geq 0$ and $\mu(x-1) = 0$. If $\mu = 0$, then $x = 2$, which is not feasible. So the constraint is active, $x = 1$, and $\mu = 2 \geq 0$. Since f and g are convex, $x^* = 1$ is the global minimum. The multiplier measures how much the constraint costs: if we relax it to $x \leq 1 + \varepsilon$, the minimum value $(1 + \varepsilon - 2)^2 = 1 - 2\varepsilon + \varepsilon^2$ decreases at the rate $\mu = 2$. For the constraint $x \leq 3$ instead, the minimum $x^* = 2$ satisfies the constraint, which is inactive, and $\mu = 0$.

Example 12.7 (Projection onto a ball)**.** Let $\mathbf{p} \in \mathbb{R}^n$ with $\|\mathbf{p}\| > 1$. We find the point of the unit ball closest to $\mathbf{p}$: minimize $\|\mathbf{x} - \mathbf{p}\|^2$ subject to $\|\mathbf{x}\|^2 - 1 \le 0$. Stationarity is $2(\mathbf{x} - \mathbf{p}) + 2\mu\mathbf{x} = \mathbf{0}$, i.e., $\mathbf{x} = \frac{\mathbf{p}}{1+\mu}$. If $\mu = 0$, then $\mathbf{x} = \mathbf{p}$ is not feasible, so the constraint is active, $\|\mathbf{x}\| = 1$, which gives $1 + \mu = \|\mathbf{p}\|$, $\mu = \|\mathbf{p}\| - 1 > 0$, and

$$\mathbf{x}^* = \frac{\mathbf{p}}{\|\mathbf{p}\|}.$$

For example, for $\mathbf{p} = (3, 4)^\top$, we get $\mathbf{x}^* = (0.6, 0.8)^\top$ and $\mu = 4$. This projection is used in projected gradient descent, which alternates gradient steps with projections onto the feasible set.

Remark 12.8 (Support vector machines). For linearly separable data $(\mathbf{x}_i, y_i)$ with labels $y_i = \pm 1$, the support vector machine finds the separating hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ with the largest margin. By the formula for the distance to a hyperplane in Lecture 2, this leads to the convex problem

$$\text{minimize } \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{subject to} \quad 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) \le 0 \quad \text{for } i = 1, \dots, N,$$

whose margin is $\frac{1}{\|\mathbf{w}\|}$. The stationarity condition gives $\mathbf{w} = \sum_i \mu_i y_i \mathbf{x}_i$, and by complementary slackness, $\mu_i > 0$ only for the points with $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$, which lie exactly on the margin. These points are the support vectors: the classifier only depends on them.

13 Key Takeaways

- A function $f : D \to \mathbb{R}^m$ on an open set $D \subseteq \mathbb{R}^n$ is differentiable at $\mathbf{x}$ if it is approximated by a linear map near $\mathbf{x}$: $f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + Df(\mathbf{x})\mathbf{h} + o(\|\mathbf{h}\|)$. The Jacobian $Df(\mathbf{x})$ is the $m \times n$ matrix of partial derivatives $\frac{\partial f_i}{\partial x_j}$; its rows belong to the outputs, its columns to the inputs. Existence of the partial derivatives alone is not enough, but continuous partial derivatives imply differentiability.

- For $f : \mathbb{R}^n \to \mathbb{R}$, the derivative is a row vector (a linear functional), and the gradient $\nabla f(\mathbf{x}) = Df(\mathbf{x})^\top$ is the vector that represents it with the dot product. The directional derivative is $\nabla f(\mathbf{x})^\top \mathbf{v}$; by Cauchy–Schwarz, the gradient points in the direction of steepest ascent, and it is orthogonal to the level sets.

- Chain rule: $D(f \circ g)(\mathbf{x}) = Df(g(\mathbf{x}))\, Dg(\mathbf{x})$, the product of the Jacobians. For a scalar function, gradients are pulled back with transposed Jacobians: $\nabla(f \circ g)(\mathbf{x}) = Dg(\mathbf{x})^\top \nabla f(g(\mathbf{x}))$. Writing the sizes under each factor is a good check.

- Important gradients: $\nabla(\mathbf{a}^\top \mathbf{x}) = \mathbf{a}$, $\nabla(\mathbf{x}^\top A\mathbf{x}) = (A + A^\top)\mathbf{x}$, $\nabla\|A\mathbf{x} - \mathbf{b}\|^2 = 2A^\top(A\mathbf{x} - \mathbf{b})$; for logistic regression and softmax with cross-entropy, the gradient with respect to the scores is "prediction minus target". For a linear layer $\mathbf{y} = W\mathbf{x} + \mathbf{b}$, $\nabla_W L = \boldsymbol{\delta}\mathbf{x}^\top$ and $\nabla_{\mathbf{x}} L = W^\top \boldsymbol{\delta}$ with $\boldsymbol{\delta} = \nabla_{\mathbf{y}} L$.

- Gradients with respect to matrices have the shape of the matrix, and satisfy $f(X+H) \approx f(X) + \langle \nabla f(X), H \rangle$ with the Frobenius inner product. Examples: $\nabla \operatorname{tr}(AX) = A^\top$, $\nabla\|AX - B\|_F^2 = 2A^\top(AX - B)$, $\nabla \log|\det X| = X^{-\top}$.

- The Jacobian determinant $|\det Df(\mathbf{x})|$ is the local factor by which f scales areas and volumes; it appears in the change of variables formula, e.g., the factor r in polar coordinates.

- Automatic differentiation applies the chain rule to the elementary operations of a program. Forward mode computes Jacobian-vector products; reverse mode computes vector-Jacobian products and gives the whole gradient of a scalar function at a small multiple of the cost of evaluating it. Backpropagation is reverse mode for neural networks: $\boldsymbol{\delta}_l = \sigma'(\mathbf{z}_l) \odot (W_{l+1}^\top \boldsymbol{\delta}_{l+1})$.
- The Hessian $\nabla^2 f(\mathbf{x})$ is the symmetric matrix of second partial derivatives. Second-order Taylor: $f(\mathbf{x}+\mathbf{h}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{h} + \frac{1}{2}\mathbf{h}^\top \nabla^2 f(\mathbf{x})\mathbf{h}$.
- At a local minimum, $\nabla f = \mathbf{0}$. At a critical point, the eigenvectors of the Hessian are the principal directions, and its eigenvalues are the curvatures along them: in these coordinates, $f \approx f(\mathbf{x}^*) + \frac{1}{2}\sum_i \lambda_i y_i^2$. All eigenvalues positive: strict local minimum (a bowl); all negative: strict local maximum; both signs: saddle point; some zero: the test is inconclusive.
- Gradient descent $\mathbf{x}_{k+1} = \mathbf{x}_k - \eta \nabla f(\mathbf{x}_k)$ decreases an L-smooth function for $\eta \le \frac{1}{L}$ and finds approximate critical points. On quadratic functions, it converges for $\eta < \frac{2}{\lambda_{\max}}$, at best with the rate $\frac{\kappa-1}{\kappa+1}$, where κ is the condition number: ill-conditioned problems zig-zag. Newton's method $\mathbf{x}_{k+1} = \mathbf{x}_k - \nabla^2 f(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k)$ converges quadratically near a minimum, but is too expensive for large models. SGD uses cheap, noisy mini-batch gradients.
- A differentiable function is convex if and only if it lies above its tangent planes, and a twice differentiable one if and only if its Hessian is positive semidefinite everywhere. For convex functions, every critical point and every local minimum is a global minimum. Least squares, logistic regression and softmax regression have convex losses; neural networks do not.
- Lagrange multipliers: at a constrained local minimum (with linearly independent constraint gradients), $\nabla f + \sum_i \lambda_i \nabla g_i = \mathbf{0}$. This gives eigenvectors for $\max_{\|\mathbf{x}\|=1} \mathbf{x}^\top A\mathbf{x}$ and the minimum norm solution $A^\top(AA^\top)^{-1}\mathbf{b}$. For inequality constraints, the KKT conditions add $\mu_i \ge 0$ and complementary slackness $\mu_i g_i(\mathbf{x}^*) = 0$, and for convex problems they are sufficient.

14 Exercises

1. (a) Describe the level sets of $f(x,y) = x^2 + 4y^2$ and of $g(x,y) = xy$, and sketch a contour plot of each function.
   (b) Let $f(x,y) = \frac{x^2 y}{x^4+y^2}$ for $(x,y) \neq (0,0)$, and $f(0,0) = 0$. Show that $f(t\mathbf{v}) \to 0$ as $t \to 0$ for every vector $\mathbf{v} \in \mathbb{R}^2$, i.e., along every line through the origin, but that f is not continuous at the origin. (Hint: look at the parabola $y = x^2$.)
2. (a) Show that the derivative of the softplus function $s(x) = \log(1+e^x)$ is the sigmoid function σ, and that s is convex.
   (b) Show that $\tanh(x) = 2\sigma(2x) - 1$, and use $\sigma' = \sigma(1-\sigma)$ to show that $\tanh' = 1 - \tanh^2$.
3. Compute the Taylor polynomial T_3 of $f(x) = \log(1+x)$ at 0, and use it to approximate $\log(1.1)$. Use Taylor's theorem to show that the error is at most $2.5 \cdot 10^{-5}$.
4. Let $f(x,y,z) = xe^{yz} + y^2 z$. Compute ∇f and the directional derivative of f at $(1,0,2)$ in the direction of the unit vector $\frac{1}{3}(1,2,2)^\top$. In which direction does f decrease fastest at this point, and at which rate?
5. Let $f(x,y) = \sqrt{|xy|}$. Show that both partial derivatives of f at the origin exist, but that f is not differentiable at the origin. (Hint: Proposition 3.5.)

6. (a) Compute the Jacobian and the Jacobian determinant of $f(x, y) = (e^x \cos y,\, e^x \sin y)^\top$.

   (b) Show that spherical coordinates $f(r, \theta, \varphi) = (r \sin\theta \cos\varphi,\, r \sin\theta \sin\varphi,\, r \cos\theta)^\top$ have the Jacobian determinant $r^2 \sin\theta$. Use the change of variables formula (Remark 4.8) to show that the ball of radius R has the volume $\frac{4}{3}\pi R^3$.

7. (a) Let $\mathbf{x}(t)$ be a differentiable curve in $\mathbb{R}^n$. Show that $\frac{d}{dt}\|\mathbf{x}(t)\|^2 = 2\mathbf{x}(t)^\top \mathbf{x}'(t)$, and conclude that if the curve lies on a sphere $\|\mathbf{x}\| = r$, then its velocity $\mathbf{x}'(t)$ is always orthogonal to $\mathbf{x}(t)$.

   (b) Let $f(u, v) = u^2 v$ and $g(x, y) = (x \cos y,\, x \sin y)^\top$. Compute $D(f \circ g)$ in two ways: as the product $Df(g(x, y))\, Dg(x, y)$, and by differentiating $f(g(x, y))$ directly.

8. (Ridge regression.) Let $X \in \mathbb{R}^{N \times d}$, $\mathbf{y} \in \mathbb{R}^N$ and $\lambda > 0$, and let $f(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|^2 + \lambda\|\mathbf{w}\|^2$. Compute ∇f and $\nabla^2 f$. Show that $X^\top X + \lambda I$ is invertible, and that the unique minimum of f is $\mathbf{w}^* = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$.

9. Let A be a symmetric $n \times n$ matrix and $R(\mathbf{x}) = \frac{\mathbf{x}^\top A \mathbf{x}}{\mathbf{x}^\top \mathbf{x}}$ for $\mathbf{x} \neq \mathbf{0}$ (the Rayleigh quotient). Show that $\nabla R(\mathbf{x}) = \frac{2}{\mathbf{x}^\top \mathbf{x}}(A\mathbf{x} - R(\mathbf{x})\mathbf{x})$, and conclude that the critical points of R are exactly the eigenvectors of A.

10. (a) Show that $\nabla_X \operatorname{tr}(AXB) = A^\top B^\top$.

    (b) Let $X \in \mathbb{R}^{N \times d}$ and $Y \in \mathbb{R}^{N \times k}$. Compute $\nabla_W \|XW - Y\|_F^2$, and show that if X has linearly independent columns, the minimum is $W^* = (X^\top X)^{-1} X^\top Y$. (This is linear regression with k outputs.)

    (c) Let X be invertible. Show that $(X + H)^{-1} = X^{-1} - X^{-1} H X^{-1} + o(\|H\|_F)$, by multiplying the right side by $X + H$. Conclude that $\nabla_X \operatorname{tr}(AX^{-1}) = -X^{-\top} A^\top X^{-\top}$.

11. Let $\mathbf{y} \in \mathbb{R}^K$ be a probability vector (with $y_k \geq 0$ and $\sum_k y_k = 1$), and let

$$\ell(\mathbf{z}) = -\sum_{k=1}^{K} y_k \log \operatorname{softmax}(\mathbf{z})_k$$

    be the cross-entropy with "soft labels". Show that $\nabla \ell(\mathbf{z}) = \operatorname{softmax}(\mathbf{z}) - \mathbf{y}$. Check that $\ell(\mathbf{z} + c\mathbf{1}) = \ell(\mathbf{z})$ for all $c \in \mathbb{R}$, and that $\nabla \ell(\mathbf{z})^\top \mathbf{1} = 0$. Why do these two facts fit together?

12. Let $f(x_1, x_2) = x_1^2 x_2 + e^{x_1 x_2}$. Draw the computation graph of f, and compute $\nabla f(1, 0)$ with forward mode (two passes) and with reverse mode (one pass), as in Example 7.1.

13. Consider the network $\hat{y} = w_2\, \sigma(w_1 x + b_1) + b_2$ with one input, one hidden unit with the sigmoid activation function, and one output, and the loss $L = \frac{1}{2}(\hat{y} - y)^2$. Derive formulas for the four partial derivatives of L with respect to w_1, b_1, w_2 and b_2, using backpropagation. Evaluate them for $x = 1$, $y = 0$, $w_1 = b_1 = b_2 = 0$ and $w_2 = 2$.

14. Compute the gradient and the Hessian of $f(x, y) = \log(1 + x + 2y)$, and its second-order Taylor approximation at the origin. Use it to approximate $f(0.1, 0.05)$, and compare with the exact value. Is the Hessian at the origin positive definite, negative definite, or neither?

15. Find and classify all critical points of $f(x, y) = x^4 + y^4 - 4xy$. Which of them are global minima?

16. Show that $f(x, y) = x^2 + y^4$ and $g(x, y) = x^2 - y^4$ have the same gradient and the same Hessian at the origin, but that the origin is a minimum of f and a saddle point of g.

17. Let $f(x_1, x_2) = x_1^2 + 9x_2^2$.

    (a) For which step sizes η does gradient descent converge for every starting point?

    (b) Find the best step size and the corresponding rate ρ. How many steps are needed to reduce the error by a factor 10^{-6}?

    (c) Substitute $x_2 = \frac{u}{3}$. How does the function look in the variables (x_1, u), and how many steps does gradient descent need now with the best step size?

18. (a) Show that Newton's method finds the minimum of $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top A\mathbf{x} - \mathbf{b}^\top\mathbf{x}$ (with A symmetric positive definite) in one step, from every starting point.

    (b) Let B be an invertible matrix, $\mathbf{c}$ a vector, and $g(\mathbf{y}) = f(B\mathbf{y}+\mathbf{c})$. Show that if $\mathbf{x}_0 = B\mathbf{y}_0 + \mathbf{c}$, then the Newton iterates for f and g satisfy $\mathbf{x}_k = B\mathbf{y}_k + \mathbf{c}$ for all k. Show by an example that this is false for gradient descent.

    (c) Apply Newton's method to $f(x) = x^4$, starting at $x_0 = 1$. Show that $x_k = (\frac{2}{3})^k$. Why is the convergence not quadratic here?

19. Which of the following functions are convex? Justify your answers. (a) $e^{x_1+x_2}$ on $\mathbb{R}^2$; (b) x_1x_2 on $\mathbb{R}^2$; (c) $\|\mathbf{x}\|_1 + \|\mathbf{x}\|^2$ on $\mathbb{R}^n$; (d) $\max(0, 1-x)$ on $\mathbb{R}$; (e) $\mathbf{x}^\top A\mathbf{x}$ on $\mathbb{R}^2$ with $A = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$; (f) $-\log(x_1x_2)$ on $\{\mathbf{x} \in \mathbb{R}^2 : x_1 > 0, x_2 > 0\}$.

20. (a) Show that if f is convex, then every sublevel set $\{\mathbf{x} : f(\mathbf{x}) \le c\}$ is convex. Give an example of a function on $\mathbb{R}$ that is not convex, although all its sublevel sets are convex.

    (b) Show that the gradient of a differentiable convex function satisfies $(\nabla f(\mathbf{x}) - \nabla f(\mathbf{y}))^\top(\mathbf{x} - \mathbf{y}) \ge 0$ for all $\mathbf{x}, \mathbf{y}$. (Hint: add two inequalities of Theorem 11.3.) What does this say for $n = 1$?

21. (Maximum entropy.) Use a Lagrange multiplier to find the probability vector $\mathbf{p} = (p_1, \ldots, p_n)$ with $p_i > 0$ and $\sum_i p_i = 1$ that maximizes the entropy $H(\mathbf{p}) = -\sum_i p_i \log p_i$. What is the maximal entropy?

22. (a) Use a Lagrange multiplier to find the point of the plane $x + 2y + 2z = 9$ that is closest to the origin. Compare with the formula for the distance to a hyperplane in Lecture 2.

    (b) Find the rectangle with sides parallel to the axes and the largest area that fits into the ellipse $\frac{x^2}{a^2} + \frac{y^2}{b^2} \le 1$.

23. Solve the problem: minimize $x^2 + y^2$ subject to $x + y \ge 2$, using the KKT conditions. Which constraint is active, and what is the multiplier? What changes for the constraint $x + y \ge -2$?

15 Programming Exercises

As in the previous lectures, implement the algorithms yourself with NumPy, and use library routines only to check your results. Most exercises also need Matplotlib for plots. Start each file with

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)  # random number generator for test data
```

P1. Numerical differentiation and its errors. Write functions `forward_diff(f, x, h)` and `central_diff(f, x, h)` for functions of one variable.

- For $f = \sin$ at $x = 1$, compute the errors of both approximations of $f'(1) = \cos 1$ for $h = 10^{-1}, 10^{-2}, \dots, 10^{-15}$, and plot them against h with `plt.loglog`.
- Explain the V-shape of the curves: for large h, the error of Proposition 2.9 dominates; for small h, rounding errors of size about $\frac{10^{-16}}{h}$ dominate. Which h is best for each method, and how accurate is the result?
- Write a function `numerical_gradient(f, X, h=1e-5)` that computes the gradient of a scalar function `f` with respect to an array `X` of any shape by central differences, one entry at a time. (Hint: loop over `np.ndindex(X.shape)`.) You will use it in the following exercises.

P2. Gradient checking. For random vectors and matrices, compare every formula of Table 1 with `numerical_gradient`, using the relative error $\frac{\|\mathbf{g}-\tilde{\mathbf{g}}\|}{\|\mathbf{g}\|+\|\tilde{\mathbf{g}}\|}$.

- Compute $\operatorname{softmax}(\mathbf{z})$ for $\mathbf{z} = (1000, 0, -1000)^\top$ directly from the definition. What goes wrong? Since $\operatorname{softmax}(\mathbf{z} - c\mathbf{1}) = \operatorname{softmax}(\mathbf{z})$, you can subtract $c = \max_i z_i$ first. Implement a stable `softmax(z)` and a stable `logsumexp(z)` in this way, and check them against `scipy.special` if you have SciPy installed.
- Deliberately introduce a mistake into one of your gradient formulas (e.g., forget a transpose) and observe the relative error.

P3. Contour plots and gradient fields. Use `np.meshgrid`, `plt.contour` and `plt.quiver` to plot level curves and gradient vectors of $x^2 + y^2$, $x^2 - y^2$, $x^3 + y^3 - 3xy$ (mark the critical points of Example 9.4) and the Rosenbrock function $(1-x)^2+100(y-x^2)^2$ (use logarithmically spaced levels). Use `plt.gca().set_aspect("equal")` so that right angles look right.

- Check Corollary 5.4 numerically: `cs = plt.contour(...)` returns an object, and `cs.get_paths()[i].vertices` contains the points of the i-th level curve. Compute the cosine of the angle between ∇f and the segments between consecutive points.
- Plot the graph of $x^2 - y^2$ as a surface. For this, create three-dimensional axes with the command `ax = plt.axes(projection="3d")`, and then use `ax.plot_surface`.

P4. Automatic differentiation. Implement reverse-mode automatic differentiation for scalar functions. Each `Var` stores its value, its parents with the local partial derivatives, and its adjoint `grad`:

```
class Var:
    def __init__(self, value, parents=()):
        self.value = value
        self.parents = parents  # pairs (parent Var, local partial derivative)
        self.grad = 0.0

    def __add__(self, other):
        other = other if isinstance(other, Var) else Var(other)
        return Var(self.value + other.value, [(self, 1.0), (other, 1.0)])

    def __mul__(self, other):
        other = other if isinstance(other, Var) else Var(other)
        return Var(self.value * other.value,
```

```
                    [(self, other.value), (other, self.value)])

def log(x):
    return Var(np.log(x.value), [(x, 1.0 / x.value)])
```

- Add subtraction, negation, division, powers with a constant exponent, `__radd__` and `__rmul__` (for expressions like `2 * x`), and the functions `exp`, `sin`, `cos` and `sigmoid`.
- Write a method `backward(self)`: list all variables that the output depends on in topological order (every variable after its parents, e.g., by a depth-first search), set the adjoint of the output to 1, and go through the list in reverse order, adding `child.grad * local` to `parent.grad` for every pair `(parent, local)`.
- Check your implementation on Example 7.1 (you should get 5.5 and 1.716), on the logistic regression loss of Example 5.6, and on random examples against `numerical_gradient`.
- (Forward mode.) Implement a class `Dual` that stores a value and a derivative $\dot{v}$, with the forward rules of Section 7, and check it on the same examples.

P5. Logistic regression. Generate two overlapping classes of points in the plane, and add a constant feature for the bias:

```
N = 200
X0 = rng.normal([-1, -1], 1.0, size=(N // 2, 2))  # class 0
X1 = rng.normal([1, 1], 1.0, size=(N // 2, 2))    # class 1
X = np.hstack([np.vstack([X0, X1]), np.ones((N, 1))])
y = np.r_[np.zeros(N // 2), np.ones(N // 2)]
```

- Implement the average cross-entropy loss, its gradient and its Hessian (Examples 5.6 and 8.4). For numerical stability, use $-y\log\sigma(z) - (1-y)\log(1-\sigma(z)) = \log(1+e^z) - yz$ and `np.logaddexp(0, z)`. Check the gradient with `numerical_gradient`.
- Run gradient descent with $\eta = 0.1$, 1 and 10, and Newton's method, until $\|\nabla L(\mathbf{w})\| < 10^{-6}$. Plot the loss against the number of iterations (with a logarithmic y-axis for $L(\mathbf{w}_k) - L(\mathbf{w}^*)$). How many iterations does each method need?
- Plot the data and the decision boundary $\mathbf{w}^\top\mathbf{x} = 0$.
- Move the class centers to $(\pm 3, \pm 3)$, so that the classes are (very likely) linearly separable, and run gradient descent again. What happens to $\|\mathbf{w}_k\|$ and to the loss, and why does the loss have no minimum? How does a ridge penalty $\lambda\|\mathbf{w}\|^2$ change this?

P6. Training a neural network. Fit $y = \sin x$ on $[-\pi, \pi]$ with a network with one hidden layer of 20 units with the tanh activation function, as in Section 7, with the loss $L = \frac{1}{2N}\sum_k (\hat{y}_k - y_k)^2$ over $N = 100$ equally spaced points.

- Implement the forward and backward passes for a whole batch, with the inputs as the rows of a matrix (Example 6.5). Check all four gradients with `numerical_gradient` on a small network.
- Initialize W_1 with standard normal entries, W_2 with normal entries with standard deviation $\frac{1}{\sqrt{20}}$, and the biases with zeros. Train with gradient descent with $\eta = 0.05$ for 20 000 steps. Plot the loss (log scale) and the fitted function.

- Build a deep network with 10 hidden layers of 20 sigmoid units (initialize the weights between hidden layers with standard deviation $\frac{1}{\sqrt{20}}$), and print the norms $\|\nabla_{W_l} L\|_F$ for all layers at initialization. What do you observe? Repeat with tanh and with ReLU.

P7. Gradient descent on quadratic functions. Let $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top A\mathbf{x}$ with $A = \mathrm{diag}(1, \kappa)$.

- For $\kappa = 10$, plot the level curves of f and the paths of gradient descent from $\mathbf{x}_0 = (8, 2)^\top$ for $\eta = 0.05$, $\frac{2}{11}$, 0.19 and 0.21, as in Figure 8.
- For $\kappa = 1, 10, 100, 1000$ and $\eta = \frac{2}{1+\kappa}$, count the steps needed to reach $\|\mathbf{x}_k\| \le 10^{-6}\|\mathbf{x}_0\|$ from $\mathbf{x}_0 = (1, 1)^\top$, and compare with Theorem 10.5.
- Implement the heavy ball method with $\eta = \frac{4}{(1+\sqrt{\kappa})^2}$ and $\beta = \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^2$, and repeat the count. How does the number of steps grow with κ now?

P8. The Rosenbrock function. Let $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$, whose minimum is $(1, 1)$, at the bottom of a long, curved valley.

- Compute ∇f and $\nabla^2 f$ by hand, and check them with `numerical_gradient`. Check that $(1, 1)$ is the only critical point, and that the Hessian there is positive definite. What is its condition number?
- Starting at $(-1.2, 1)$, run gradient descent with $\eta = 10^{-3}$ until $\|\mathbf{x}_k - (1, 1)^\top\| < 10^{-4}$. How many steps does it need? What happens for $\eta = 3 \cdot 10^{-3}$? (Look at consecutive iterates, and compare η with $\frac{2}{\lambda_{\max}}$ for the Hessian at $(1, 1)$.)
- Implement gradient descent with a backtracking line search: in each step, start with $\eta = 1$ and halve it until $f(\mathbf{x} - \eta\nabla f(\mathbf{x})) \le f(\mathbf{x}) - 10^{-4}\eta\|\nabla f(\mathbf{x})\|^2$.
- Run Newton's method from the same starting point. Plot the paths of all three methods on a contour plot, and compare the numbers of steps.

P9. Stochastic gradient descent. Generate a linear regression problem with many data points:

```
N, d = 10_000, 20
X = rng.normal(size=(N, d))
w_true = rng.normal(size=d)
y = X @ w_true + 0.5 * rng.normal(size=N)
```

and let $L(\mathbf{w}) = \frac{1}{N}\|X\mathbf{w} - \mathbf{y}\|^2$.

- Compute the minimum $\mathbf{w}^*$ with `np.linalg.lstsq`, and the eigenvalues of the Hessian $\frac{2}{N}X^\top X$. Which step sizes are safe for gradient descent?
- Compare gradient descent ($\eta = 0.1$) with SGD with mini-batches of size 1, 32 and 256 (draw the indices with `rng.integers(0, N, size=B)`), with step sizes 0.01, 0.05 and 0.1. Plot $L(\mathbf{w}_k) - L(\mathbf{w}^*)$ against the number of processed data points (one gradient step of full gradient descent processes N points).
- With a constant step size, SGD stops improving at some level. Try the decreasing step sizes $\eta_k = \frac{\eta_0}{1+k/1000}$ and compare.

P10. Constrained optimization.

- (Projected gradient ascent.) For a random symmetric positive semidefinite matrix $A = B^\top B$, maximize $\mathbf{x}^\top A\mathbf{x}$ on the unit sphere by repeating $\mathbf{x} \leftarrow \frac{\mathbf{x}+\eta A\mathbf{x}}{\|\mathbf{x}+\eta A\mathbf{x}\|}$. Compare the result with the top eigenvector from `np.linalg.eigh` (Example 12.3). What happens as $\eta \to \infty$? (Compare with power iteration from Lecture 1.)

- (Projected gradient descent.) Let `A = rng.normal(size=(30, 10))`, let `x_true` be a random vector of norm 3, and `b = A @ x_true + 0.1 * rng.normal(size=30)`. Minimize $\|A\mathbf{x} - \mathbf{b}\|^2$ subject to $\|\mathbf{x}\| \leq 1$, by alternating gradient steps with step size $\frac{1}{L}$, where $L = 2\|A\|_2^2$, with the projection of Example 12.7. At the result, compute the multiplier μ from the stationarity condition, and check all four KKT conditions.
- (Minimum norm solutions.) For a random 3×6 matrix A and $\mathbf{b} \in \mathbb{R}^3$, compute $\mathbf{x}^* = A^\top (AA^\top)^{-1}\mathbf{b}$ (Example 12.4) and compare it with `np.linalg.pinv(A) @ b`. Get a basis of null(A) from the SVD (the last rows of $V^\top$), and check that adding a null space vector to $\mathbf{x}^*$ gives another solution of $A\mathbf{x} = \mathbf{b}$ with a larger norm.